ADAPTIVE COMPONENT-LEVEL MODEL LOADING FOR RESOURCE-CONSTRAINED BROWSER-BASED AI INFERENCE

MINI PROJECT REPORT

Submitted by

[STUDENT NAME]
[REGISTER NUMBER]

Under the Guidance of

[GUIDE NAME]
([DESIGNATION], Directorate of Online Education)

in partial fulfillment for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

[SRM ONLINE LOGO]

DIRECTORATE OF ONLINE EDUCATION
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR – 603 203

[MONTH YEAR]

DIRECTORATE OF ONLINE EDUCATION
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR – 603 203

BONAFIDE CERTIFICATE

This Mini Project Work Report titled “Adaptive Component-Level Model Loading for Resource-Constrained Browser-Based AI Inference” is the bonafide work of [STUDENT NAME] [REGISTER NUMBER], who carried out the Mini Project Work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other internship report or dissertation based on which a degree or award was conferred on an earlier occasion on this or any other candidate.

[SIGNATURE]
[GUIDE NAME]
[DESIGNATION]
Directorate of Online Education
SRM Institute of Science and Technology

[SIGNATURE]
[NAME]
[HEAD / DIRECTOR — DESIGNATION]
Directorate of Online Education
SRM Institute of Science and Technology

Date: [DATE]

[SEAL]

Abstract

Transformer language models are increasingly run inside the web browser so that text never leaves the user's device. A browser tab, however, is a constrained environment: in this project a 255.5 MiB DistilBERT sentiment model needed a 656 MiB WebAssembly heap, and that heap was not returned when the model was released. The objective of this project was to design, build and evaluate a library that loads such a model as separate components and decides, for each device, which components to keep in memory, at which numeric precision, and when to unload them, while keeping accuracy above a stated floor.

The fine-tuned DistilBERT-SST2 model was split into four ONNX components (embeddings, two pairs of encoder layers, and the last two layers with the classifier), exported in fp32 and dynamic int8, and given two early-exit classifiers. In the browser each component runs in its own Web Worker on ONNX Runtime Web (WebAssembly); in a cloud measurement, terminating the Workers brought the page's measured memory back to its starting level, whereas releasing the model sessions freed none of it. Components are cached in IndexedDB under their SHA-256 hash and verified on every read. A deterministic decision engine reads the browser's resource signals or a simulated device profile, rejects plans that would exceed the memory budget or the accuracy floor, and selects the plan with the lowest weighted cost of memory, time and initial load, together with a written explanation. The work is packaged as the npm-style package `adaptive-onnx-loader` with a demonstration page.

Pre-registered experiments were run on a cloud machine and, separately, on an Apple Silicon Mac. Loading one component at a time reduced peak memory by 55–57% against the unsplit model (cloud), at the cost of a 50–72% longer time to first inference; in the cloud the cache made revisits 1.16 to 78 times faster depending on link speed; and early exit reduced single-request latency in resident plans by 18.6–20.4% in the cloud and by 21.0–21.2% on the Mac, with an accuracy loss (0.11–0.34 points) that was not statistically significant. Several expectations were not met: pre-starting the next component did not shorten time to first inference, no precision difference was significant on the 872-sentence validation set, and the engine's time model was optimistic for plans with pre-start or pinned components (28–52% in the cloud, 15–29% on the Mac), which led to a 24–43% regret for one cloud profile. These results and their limitations are reported without adjustment.

Acknowledgement

[ACKNOWLEDGEMENT — to be written by the student.]

Contents

Abstract ii

Acknowledgement iii

List of Figures vi

List of Tables viii

List of Abbreviations x

1 Introduction 1

1.1 Background and motivation 1
1.2 Problem statement 1
1.3 Objectives 1
1.4 Scope 2
1.5 Organisation of the report 2

2 Theoretical Background 4

2.1 Transformers and DistilBERT 4
2.2 ONNX and ONNX Runtime Web 5
2.3 Quantization and mixed precision 5
2.4 Component-level model splitting 6
2.5 WebAssembly, threads and cross-origin isolation 7
2.6 Web Workers and transferable objects 7
2.7 IndexedDB, storage quota and content addressing 8
2.8 Adaptive resource management and decision policies 8
2.9 Early exit 9
2.10 Project requirements 10
2.11 Software development methodology 11

3 Practical Implementation 13

3.1 System overview 13
3.2 System architecture 14
3.3 Model preparation and splitting 16
3.4 ONNX export, quantization and validation 18
3.5 Browser baseline inference 20
3.6 Worker-based component execution 22
3.7 Execution policies 26
3.8 IndexedDB caching and memory management 29
3.9 Resource monitor and decision engine 32
3.10 Early exit and mixed precision 36
3.11 Package and public API 37
3.12 Testing and verification 40
3.13 Experimental evaluation 40
3.13.1 Design 41
3.13.2 Pilot and execution 42
3.13.3 Part A: cloud and offline results 43
3.13.4 Part B: Mac results 48
3.13.5 Verdicts 51
3.13.6 Unsupported hypotheses, deviations and unexplained observations 52

4 Conclusion.....54
4.1 Outcome.....54
4.2 Major findings.....54
4.3 Limitations.....55
4.4 Future enhancements.....56
References.....57
Appendix A Findings register.....60
Appendix B Detailed Phase 7 statistics (Part A: cloud and offline).....63
Appendix C Detailed Mac results (Part B).....69
Appendix D Experiment configurations.....71
Appendix E Raw-data index.....73
Appendix F Repository structure and reproduction.....74

List of Figures

Figure 1. Structure of the DistilBERT sequence classifier used in this project (drawn from python/adaptive_split/chunks.py).....5
Figure 2. The four model components, the tensors that cross each boundary, and their parameter counts (from Phase 1 report §3, Step 1).....7
Figure 3. Early-exit decision on the component chain (logic as implemented in workers/planRunner.js).....10
Figure 4. Phase-gated incremental life cycle: each phase ends at an approval gate (Master Plan §4).....12
Figure 5. Flow of a loadModel() call and later infer() calls (from src/api.js and Phase 6 report §2).....14
Figure 6. System architecture: offline pipeline, artefacts, page-side package modules and per-component Workers.....15
Figure 7. Component definitions: the split specification and the layer-group module.....17
Figure 8. Offline model pipeline (python/run_phase1.sh).....18
Figure 9. Dynamic int8 quantization of each exported graph.....19
Figure 10. Manifest entry for component C2 (fp32 part shown first, then the start of the int8 part).....20
Figure 11. WebAssembly-heap probe used as the primary memory metric.....21
Figure 12. Baseline page (demo/phase2-baseline.html) after loading the int8 model and classifying one sentence.....22
Figure 13. Message sequence between the page and one component Worker (protocol from the header of workers/chunkWorker.js).....23
Figure 14. Worker protocol and message handling; output buffers are returned in the transfer list.....24
Figure 15. Phase 3 test page (demo/phase3-chunks.html) after experiment B, resident Workers, int8, 100 sentences.....26
Figure 16. Which component Workers are alive over time in each plan shape (schematic, not measured; semantics of PlanRunner.classifyMany).....27
Figure 17. Core loop of PlanRunner: spawn or reuse, pre-start, run, early exit, terminate unless pinned.....28
Figure 18. Tiered component loading with integrity checks (cache/chunkLoader.js, workers/workerPool.js; Phase 4 report §4.1).....29
Figure 19. IndexedDB read with verify-on-read, fall-through to the network, verify-before-store.....30
Figure 20. IndexedDB schema of the component store (from ChunkStore.open() in cache/chunkStore.js).....30
Figure 21. Cache test page (demo/phase4-cache.html) after experiment F1, one request run twice: the first fills the cache from the network, the second reads IndexedDB. Documentation capture, CLOUD-HL, localhost.....31
Figure 22. Application-generated view of the IndexedDB store after the documentation captures (five entries: the four int8 components and C2 in fp32 for the mixedC2 plan).....32
Figure 23. The seven steps of decide() (engine/planner.js).....33
Figure 24. Hard constraints first, then the min-normalised weighted cost and the deterministic order.....34

Figure 25. Decision-engine page (demo/phase5-engine.html), mid profile, default timings: the six-line explanation and the start of the decision record.....35
Figure 26. High profile with early exit requested: early-exit plans are rejected by the accuracy floor and the explanation says so.....36
Figure 27. Minimal use of the public API, from the package README.....38
Figure 28. Demonstration page (demo/index.html) with the mid profile after classifying three sentences: live state, explanation, alternatives, cache and per-profile comparison.…39
Figure 29. Cloud: peak memory by execution shape and precision (E1, results/phase7/summary/analysis.json, E1.table[*].peakTotalMiB).....44
Figure 30. Cloud: latency with early exit off and on (E1 block, analysis.json, E3latency).....45
Figure 31. Cloud: cache revisit speed-up by link speed (analysis.json, E5[*].speedup, speedupCI95).....46
Figure 32. Cloud: engine time-prediction error by plan shape and cache state (analysis.json, ENG.timeErrorByShapeAndCache).....48
Figure 33. Mac: single-request latency against ORT page threads, int8 and fp32 (MAC-1; analysis_mac_final.json, MAC1.table).....49
Figure 34. Mac: latency with early exit off and on (MAC-3 re-run; analysis_mac_final.json, MAC3.table).....50
Figure 35. Mac: engine time-prediction error by plan shape and cache state (MAC-2 re-run; analysis_mac_final.json, MAC2.timeErrorByShape).....51

List of Tables

Table 1. Abbreviations used in this report.....x
Table 2. Hardware and runtime environments.....10
Table 3. Software stack (versions pinned for reproducibility).....11
Table 4. Functional and non-functional requirements.....11
Table 5. Phases, increments and approvals.....12
Table 6. Package modules and their responsibilities (packages/adaptive-onnx-loader/src/).....15
Table 7. Model components and file sizes (Phase 1 report §2, “Artifacts”).....17
Table 8. Offline validation in native ONNX Runtime, batch 32 (Phase 1 report §3, Step 6).....19
Table 9. Browser baseline, unsplit model (CLOUD-HL-1T for 1 thread, CLOUD-HL-2T for 2 threads; 872 sentences, batch 1; Phase 2 report §2).....21
Table 10. Memory-release proof (CLOUD-HL-1T, measureUserAgentSpecificMemory; Phase 3 report §2).....25
Table 11. Peak WebAssembly memory by execution shape (CLOUD-HL-1T, 872 sentences; Phase 3 report §2).....28
Table 12. Simulated device profiles (engine/profiles.js).....32
Table 13. Plans chosen by the engine (cloud calibration; Phase 5 report §3.1).....35
Table 14. Validation accuracy used by the hard constraint (WASM engine, 872 sentences, τ = 0.95).....36
Table 15. Public API (index.d.ts, src/api.js).....37
Table 16. AdaptiveLoaderError codes (Phase 6 report §2).....37
Table 17. Verification by phase.....40
Table 18. Pre-registered hypotheses and predictions (experiment plan v1.2 §6).....41
Table 19. Experiment configurations (bench/configs.json; results report header).....42
Table 20. Execution summary, Part A (cloud and offline; results report §1).....43
Table 21. Early-exit accuracy on 872 validation sentences (results report §4.1).....44
Table 22. Cache: TTFI cold → revisit, median, speed-up [95% CI] (results report §6).....45
Table 23. Cloud: engine time-prediction error by plan shape, median [95% CI]; negative = optimistic (results report §11.2).....47
Table 24. Mac validation of the combined data (Mac results §1).....48
Table 25. Mac: engine time-prediction error by plan shape, median [95% CI] (Mac results §5.3). 50
Table 26. Verdicts, cloud and Mac in separate columns (results report §15).....51
Table 27. Results that did not support the hypotheses, deviations and unexplained observations. 52
Table 28. Findings register (condensed).....60
Table 29. E1: peak memory, TTFI and M4 latency by policy and precision (CLOUD-HL-1T, cache off).....63
Table 30. E3: early-exit latency and compute (CLOUD-HL-1T).....64
Table 31. E3: bytes for single cold requests by exit class (CLOUD-HL-1T).....64
Table 32. E2: pre-start, TTFI and 100-sentence workload (CLOUD-HL-1T-NET-*).....64
Table 33. E4: all 16 precision combinations (NODE-WASM-X86, 872 sentences).....65
Table 34. E-B: int8 batch dependence, flips vs batch 1 (of 872) and accuracy.....66
Table 35. F-5: native vs WASM at batch 1.....66
Table 36. E-F28: session creation time by state (400 creations, n = 40 per cell).....66

Table 37. E-ENG: decisions, rank and regret (CLOUD-HL-1T).....67
Table 38. E-ENG: I-5.1 refit, proposal only (not applied).....67
Table 39. Deviations and unexpected findings, Part A.....67
Table 40. Mac result files and their use.....69
Table 41. MAC-1: thread sweep (original run), baseline, w100.....69
Table 42. MAC-3: policies and early exit (re-run).....69
Table 43. MAC-2: decisions compared with the cloud’s.....70
Table 44. MAC-2: regret and rank (top-4 predicted plans only).....70
Table 45. MAC-5: F-14 fetch diagnostic, six rounds.....70
Table 46. Registered configurations.....71
Table 47. Phase 7 raw-data index (results report §17).....73
Table 48. File counts in results/phase7/ at the time of writing (proposed Phase 7 commit).....73
Table 49. Repository structure (top level).....74

List of Abbreviations

Table 1. Abbreviations used in this report.

Abbreviation	Meaning
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
CDP	Chrome DevTools Protocol
CI	Confidence interval (95%, bootstrap, unless stated)
COOP / COEP	Cross-Origin-Opener-Policy / Cross-Origin-Embedder-Policy (HTTP headers)
CPU	Central Processing Unit
EP	Execution provider (ONNX Runtime back-end)
fp32	32-bit floating point
G	Group size: sentences processed per component load
IDB	IndexedDB (browser object database)
int8	8-bit integer (here: dynamic int8 quantization)
L	Initial load time on the first request’s critical path (engine term)
LRU	Least recently used
M	Predicted peak memory (engine term)
M4	Latency metric: single-request median, or amortised time per sentence for chunk-major plans
MiB / MB	Mebibyte (2^{20} bytes) / megabyte (10^{6} bytes)
mixedC2	Precision plan: component C2 in fp32, all others int8
ONNX	Open Neural Network Exchange
ORT	ONNX Runtime
SDLC	Software Development Life Cycle
SHA-256	Secure Hash Algorithm, 256-bit
SST-2	Stanford Sentiment Treebank, binary task (GLUE)
T	Predicted time per sentence (engine term)
TTFI	Time to first inference
UA	User agent (browser)
vCPU	Virtual CPU
WASM	WebAssembly
τ (tau)	Early-exit confidence threshold

1 Introduction

1.1 Background and motivation

Running a machine-learning model directly in the web browser has practical advantages. The user's text is processed on the user's own device, no inference server has to be paid for or maintained, and the application keeps working when the network is slow. Toolkits such as ONNX Runtime Web make this possible by executing a model compiled to WebAssembly inside the page (ONNX Runtime 2026a). The obstacle is resources. A browser tab shares the device's memory with every other tab and application, the model file must be downloaded before the first prediction, and devices range from high-end laptops to low-memory phones.

Two measurements made early in this project show how severe the problem is for a whole model loaded at once. First, the DistilBERT sentiment model used here is a 255.5 MiB file in fp32, but after loading it occupied **656 MiB** of WebAssembly memory; the 64.2 MiB int8 version occupied 180 MiB (Phase 2 report §2; finding F-3). Second, the WebAssembly heap **did not shrink** after the model session was released, either in the cloud browser or on a real Mac (F-4). Within one page, "unloading" a model therefore did not reduce the WebAssembly heap. These observations motivated the approach of this project: treat the model as a set of components that can be loaded, run and discarded individually, and choose automatically how to do so on each device.

1.2 Problem statement

The project's research question, as fixed in the Master Plan (§1), was: *can exposing a transformer model's components as independently managed execution units reduce memory use and transfer overhead in the browser, through adaptive loading and precision selection, while keeping accuracy within an acceptable bound?*

Stated as an engineering problem: given a transformer classification model and a device whose memory, processor and network capacity are only partly known to the web page, how can the page load and run the model so that peak memory stays within a budget, accuracy stays above a stated floor, and latency is as low as those constraints allow? The project addresses this for one model and task, the DistilBERT model fine-tuned on SST-2 sentiment classification (Sanh et al. 2019; Socher et al. 2013), executed with ONNX Runtime Web on the WebAssembly back-end.

1.3 Objectives

1. Split the model into components without changing its outputs, and verify the split numerically.

2. Execute each component in its own Web Worker, so that its memory can be released by terminating the Worker.
3. Cache components persistently in the browser with integrity verification, so that repeat visits do not download the model again.
4. Build a deterministic decision engine that selects an execution plan (precision, which components stay loaded, group size, pre-start, early exit) for a device profile and explains its choice.
5. Offer early exit and mixed precision as options that trade a measured amount of accuracy for speed or size.
6. Package the work as a reusable JavaScript package with a documented API and a demonstration page.
7. Evaluate the system with pre-registered hypotheses on two machines (a cloud machine and a real Mac), reporting the two separately, including results that do not support the hypotheses.

1.4 Scope

In scope: one model (DistilBERT-SST2), one task (binary sentiment classification), three precision settings (fp32, int8, and mixed precision with one fp32 component), the ONNX Runtime Web WebAssembly back-end in Chromium-based browsers, a four-component split, IndexedDB caching, simulated device profiles (low, mid, high) and the browser's own signals, early exit with two exit heads (Master Plan §2). The evaluation used a 2-vCPU cloud machine and an Apple Silicon Mac whose browser reports 8 logical processors (Section 2.10).

Out of scope (Master Plan §2): generative or causal models, which would need a key–value cache to be handed between components; the WebGPU back-end, whose int8 operator support was inconsistent and would have mixed two back-ends in the comparisons; and training or fine-tuning the base model (only the two small early-exit heads were trained). In addition, the evaluation did not include a real low-memory device or a real network; these are limitations of the results and are listed in Section 4.3.

1.5 Organisation of the report

The remainder of the report is organised in four parts, following the structure of the departmental sample report:

- **Chapter 2, Theoretical background**, introduces the transformer and DistilBERT, ONNX and ONNX Runtime Web, quantization, model splitting, WebAssembly, Web Workers, IndexedDB, decision policies and early exit, and states the project requirements and the development methodology.

- **Chapter 3, Practical implementation**, describes the system architecture and each component as built, with source-code excerpts, diagrams and screenshots, followed by the experimental method and the results. Cloud results and Mac results are always reported separately.
- **Chapter 4, Conclusion**, summarises the outcome and the main findings, lists the limitations and proposes future work.
- **Appendices** contain the findings register, detailed statistics, the experiment configurations, the raw-data index and reproduction instructions.

Throughout the report, file paths are written in `monospace`; MiB means 2^{20} bytes and MB means 10^6 bytes; a value written as “median [a, b]” is a median with its 95% bootstrap confidence interval; and every measurement is labelled with the configuration it comes from (for example CLOUD-HL-1T or MAC-CHROME-T4).

2 Theoretical Background

This chapter introduces the concepts on which the implementation in Chapter 3 is built. Each section first explains the general idea, with a reference to its original source, and then states briefly how the project uses it. Section 2.10 lists the project requirements and Section 2.11 describes the development methodology that was actually followed.

2.1 Transformers and DistilBERT

The transformer is a neural-network architecture built from stacked attention layers (Vaswani et al. 2017). In an encoder layer, every token's vector is updated by **multi-head self-attention**, which lets each token weigh information from every other token in the sentence, followed by a position-wise feed-forward network; each of the two sub-layers is wrapped in a residual connection and layer normalisation. Because a layer maps a sequence of vectors to a sequence of vectors of the same shape, layers can be stacked, and the output of one layer is the only thing the next layer needs, apart from the attention mask that marks padding positions.

BERT applies a stack of transformer encoder layers to text and is pre-trained on unlabelled text before being fine-tuned for a specific task (Devlin et al. 2019). For classification, the vector of a special `[CLS]` token at the start of the input is passed to a small classifier. DistilBERT is a smaller BERT obtained by knowledge distillation: it keeps the same hidden size (768) but has 6 instead of 12 layers, and its authors report that it retains 97% of BERT's language-understanding performance while being 40% smaller and 60% faster (Sanh et al. 2019).

This project uses the publicly released checkpoint `distilbert-base-uncased-finetuned-sst-2-english` (Hugging Face 2026), which is DistilBERT fine-tuned on SST-2, the binary sentiment task of the Stanford Sentiment Treebank (Socher et al. 2013) as distributed in the GLUE benchmark (Wang et al. 2018). The model has **66,955,010 parameters**, and on the 872-sentence SST-2 validation set it reaches **91.06%** accuracy in PyTorch (794 / 872), which the Phase 1 report notes matches the published figure for this checkpoint (Phase 1 report §2). Its structure is shown in Figure 1.

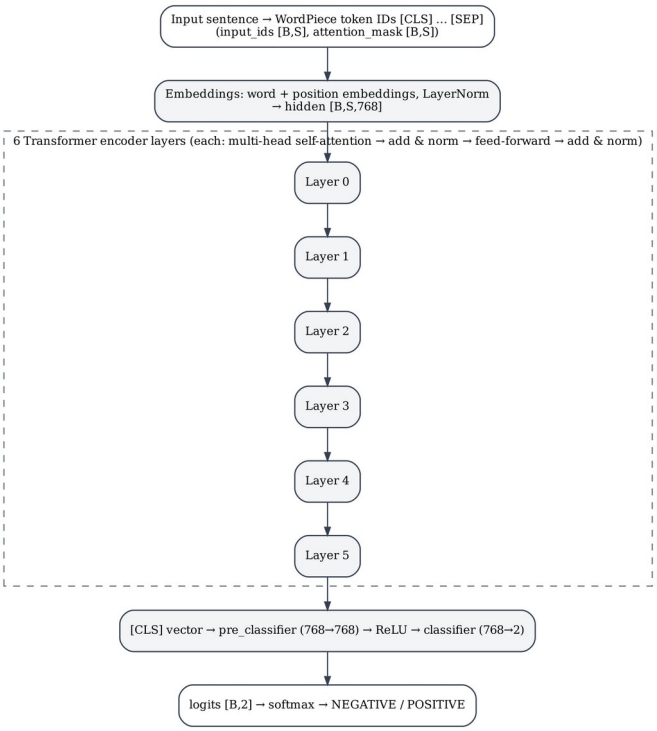


Figure 1. Structure of the DistilBERT sequence classifier used in this project (drawn from `python/adaptive_split/chunks.py`).

2.2 ONNX and ONNX Runtime Web

ONNX (Open Neural Network Exchange) is an open format that describes a trained model as a computation graph of standard operators together with its weights, so that a model trained in one framework can be executed by another (ONNX 2026). Each ONNX file declares an **operator set (opset)** version, which fixes the semantics of its operators. PyTorch can export a module to ONNX by tracing it with example inputs (PyTorch 2026).

ONNX Runtime (ORT) executes ONNX graphs. ONNX Runtime Web is its JavaScript distribution for browsers; it runs the graph through an **execution provider**, and the one used here is the WebAssembly (WASM) execution provider, which runs on the CPU (ONNX Runtime 2026a). A model is loaded into an `InferenceSession`; `session.run(feeds)` takes named input tensors and returns named output tensors; `session.release()` frees the session's resources. In this project every model component is a separate ONNX graph (opset 17) and therefore a separate `InferenceSession`, and the exact runtime version is pinned to `onnxruntime-web` 1.20.1 so that operator behaviour is the same in every run (Phase 0 report §3).

2.3 Quantization and mixed precision

Quantization stores weights with fewer bits. In integer quantization a real value w is represented by an 8-bit integer q with a scale and a zero point, $w \approx scale \times (q - zero_point)$, which reduces weight storage from 4 bytes to 1 byte (Jacob et al. 2018). In **dynamic**

quantization, as provided by ONNX Runtime's `quantize_dynamic`, the weights are quantized once, ahead of time, while the activations are quantized at run time from their observed range, so no calibration data set is needed (ONNX Runtime 2026b).

In this project dynamic int8 quantization (`QInt8`, operators `MatMul` and `Gather`) reduced the model from 267.9 MB to 67.3 MB, a factor of 3.98 (Phase 1 report §2). Quantization does not affect all parts of a network equally: when only one component at a time was quantized, the third component (layers 2–3) changed the most predictions (20 of 872), and the embedding component the fewest (4 of 872) (finding F-2). This motivated a **mixed-precision** setting, called *mixedC2* in this report, in which that one component is kept in fp32 and the other three are int8.

A side effect of dynamic quantization matters for evaluation. Because the activation range is computed per input tensor, a batch of several sentences shares one scale, so an int8 prediction can depend on which other sentences are in the same batch (finding F-6). For this reason the system always runs one sentence per inference call, and the batch effect was measured separately as experiment E-B (Section 3.13).

2.4 Component-level model splitting

Splitting a network into parts that run separately is an established idea. Neurosurgeon, for example, partitions a deep network at layer granularity between a mobile device and a cloud server and chooses the partition point automatically (Kang et al. 2017). This project applies layer-group splitting **within one device**: the parts are not distributed between machines but are loaded, executed and discarded one by one in the browser, so that only the parts currently needed occupy memory.

A split is only acceptable if it is **lossless**, meaning that running the parts in sequence gives the same outputs as the unsplit model. For a transformer encoder this holds when every cut is placed between layers and the hidden-state tensor and attention mask are passed across the cut unchanged. The project uses four components: embeddings (C0), layers 0–1 (C1), layers 2–3 (C2), and layers 4–5 with the classifier (C3), as shown in Figure 2. Placing the 23.8 million embedding parameters in their own component balances the components and isolates the part that is least sensitive to int8 (Phase 1 report §4).

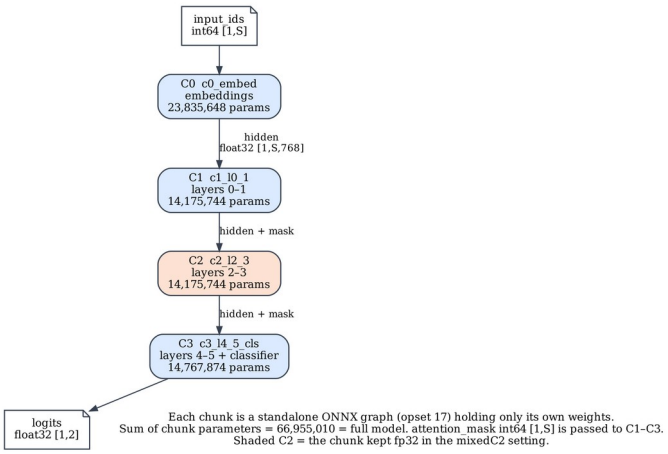


Figure 2. The four model components, the tensors that cross each boundary, and their parameter counts (from Phase 1 report §3, Step 1).

2.5 WebAssembly, threads and cross-origin isolation

WebAssembly is a portable binary instruction format that browsers execute at near-native speed in a sandbox (Haas et al. 2017). A WebAssembly module works on a **linear memory**: a single contiguous, resizable array of bytes. The memory can be grown but there is no operation to shrink it. A library that allocates a large model inside WebAssembly can free that space for its own reuse, but the linear memory itself, and therefore the memory the browser holds for it, stays at its peak size for as long as the instance exists. This was observed directly for ORT in this project (F-4).

Multi-threaded WebAssembly uses a `SharedArrayBuffer` as its memory. Browsers allow shared memory only in pages that are **cross-origin isolated**, which a server enables by sending two HTTP headers, `Cross-Origin-Opener-Policy: same-origin` and `Cross-Origin-Embedder-Policy: require-corp`; a page can check the result through the `crossOriginIsolated` property (MDN Web Docs 2026c). The project's development server sends both headers, and the library refuses to start in a page that is not isolated (Phase 2 report §3.2; Phase 6 report §2).

2.6 Web Workers and transferable objects

A Web Worker runs a script on a background thread with its own global scope; the page and the Worker communicate only by passing messages with `postMessage` (MDN Web Docs 2026a). Because a Worker has its own JavaScript realm, a WebAssembly instance created inside it, and that instance's linear memory, belong to the Worker. When the page calls `worker.terminate()`, the Worker and everything it owns are destroyed. This is the property the project relies on: since a WASM heap cannot shrink, the project releases a component's memory by running the component in its own Worker and terminating that Worker. The effect was measured in the cloud browser with the browser's page-level

memory API (Section 3.6); whether, and when, the operating system reclaims that memory was not measured.

Messages are normally copied with the structured-clone algorithm. Certain objects, including `ArrayBuffer`, can instead be **transferred**: ownership of the underlying memory moves to the receiver without copying, and the sender's reference becomes unusable (MDN Web Docs 2026b). A hidden-state tensor for one 128-token sentence is 0.375 MiB (128 × 768 × 4 bytes; Phase 5 report §5.3). Hidden states are sent between the page and the component Workers as transferred buffers, which kept the messaging overhead at 1.8–2.6% of the time per sentence (Phase 3 report §5.2).

2.7 IndexedDB, storage quota and content addressing

IndexedDB is a transactional object database built into browsers. It stores structured values, including large binary `ArrayBuffers`, in named **object stores** inside a database, and it is available both in pages and in Workers (MDN Web Docs 2026d). Each database belongs to one **origin** (scheme, host and port), so a page served from a different port sees a different, empty database; this caused a measurement problem in Phase 7 and is recorded as F-36. The space available is governed by the browser's storage quota, which a page can query with `navigator.storage.estimate()`; unless storage has been marked persistent, the browser may evict it under storage pressure (MDN Web Docs 2026f). In the headless cloud browser the reported quota was about 10 GiB and `persist()` returned false (F-26).

Content addressing stores each item under a cryptographic hash of its bytes. The project uses the SHA-256 digest computed with the Web Crypto API's `SubtleCrypto.digest()` (MDN Web Docs 2026e). The hash of every component file is recorded at build time in the model manifest. Using the hash as the cache key means that a changed model file automatically gets a new key, and comparing a stored item's hash with the manifest detects corruption. Hashing ran at 0.4–1.1 GB/s in the cloud and 1.2–2.0 GB/s on the Mac (F-25; Phase 4 report §13.3).

2.8 Adaptive resource management and decision policies

An adaptive loader has to decide how to run the model with incomplete knowledge of the device. Browsers expose a few coarse signals: `navigator.deviceMemory` reports an approximate amount of device memory in GiB, deliberately rounded and capped to limit fingerprinting (MDN Web Docs 2026g; W3C 2026); `navigator.hardwareConcurrency` reports the number of logical processors available (MDN Web Docs 2026h); and the Network Information API, where supported, reports an estimated downlink bandwidth and round-trip time (MDN Web Docs 2026i). These are **browser-reported signals, not hardware specifications**. For example, Chrome on the Mac used in this project reported `deviceMemory` = 16 and `hardwareConcurrency` = 8, and its user-agent string reported

“Intel Mac OS X 10_15_7” on an Apple Silicon (arm64) machine (F-16). The headless cloud browser reported `deviceMemory` = 8 and `hardwareConcurrency` = 2 (Figure 15 and the `hardwareConcurrency` / `deviceMemory` fields of the result rows), on a machine whose operating system reported 2 CPUs and 7.8 GiB of memory (`results/phase7/*/env_*.json`).

The project’s decision policy follows a common two-stage pattern for constrained choices. **Hard constraints** (memory budget, accuracy floor) first remove every infeasible plan; then a **weighted cost** ranks the remaining plans. Because memory (MiB), time per sentence (ms) and initial load (ms) have different units, each term is divided by the best value among the feasible plans before the weights are applied (a fourth network term uses (N + 1) / (N_min + 1), and its weight is 0 in all three profiles), so a weight expresses relative importance (“memory matters three times as much as latency”) rather than a unit conversion (Phase 5 report §7). The policy is deterministic: the same inputs always produce the same plan, which makes it testable and explainable.

2.9 Early exit

Early exit adds small classifiers after intermediate layers. At inference time, if an intermediate classifier is confident enough, the prediction is returned without running the remaining layers. BranchyNet introduced this for convolutional networks, using the entropy of the side-branch output as the exit criterion (Teerapittayanon et al. 2016), and DeeBERT applied it to BERT-style transformers (Xin et al. 2020). The threshold controls the trade-off: a lower threshold lets more inputs exit early, which saves computation, but the early classifiers are less accurate than the full model.

In this project two exit heads were trained with the backbone frozen: E1 after C1 (2 layers) and E2 after C2 (4 layers). A sentence exits when the maximum softmax probability of a head is at least τ (Figure 3). The threshold τ = 0.95 was chosen **on a development split only** (2,000 training sentences held out from head training) as the smallest τ that agrees with the full model on at least 99% of that split; the validation set was used only for reporting (Phase 5 report §3.3). Early exit is off by default and changes predictions, so a plan with early exit must still satisfy the profile’s accuracy floor.

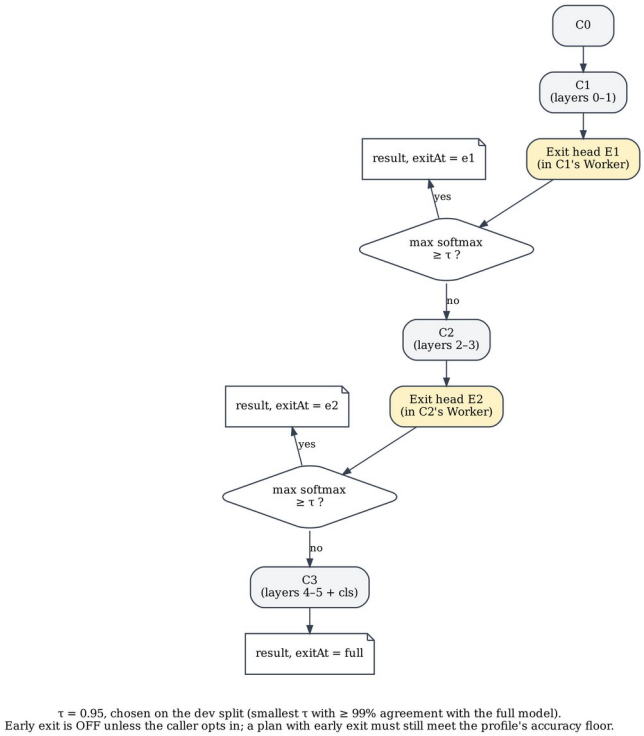


Figure 3. Early-exit decision on the component chain (logic as implemented in `workers/planRunner.js`).

2.10 Project requirements

Table 2 lists the machines used. The distinction between **operating-system-reported** values and **browser-reported** values is deliberate: browsers round, cap or freeze some values, so the browser's view of a device is not its physical specification. Values that were not recorded are marked as such rather than estimated. Table 3 lists the software stack and Table 4 the requirements that the system was built to meet.

Table 2. Hardware and runtime environments.

Environment	Used for	OS-reported	Browser-reported	Source
Cloud workspace (CLOUD-HL-1T*, NODE-WASM-X86, NATIVE-X86-*)	All Part A browser experiments, Node and native x86 runs	Linux x86-64, 2 CPUs, "Intel(R) Xeon(R) Processor", total memory 7.8 GiB	Headless Chromium 153: hardwareConcurrency 2, deviceMemory 8	OS: `results/phase7/*/env_*.json`; browser: result rows, Figure 15
Apple Silicon Mac (MAC-CHROME-*)	Part B browser experiments, manual checks	macOS 26.7.1, arm64; physical memory and CPU model not recorded	Chrome 155: hardwareConcurrency 8, deviceMemory 16; UA string reports "Intel Mac OS X 10_15_7"	Phase 0 report §3; `analysis_mac_final.json` (env); F-16
Linux sandbox on the Mac (NATIVE-ARM64-*)	Python pipeline, unit tests, native aarch64 runs	Ubuntu 22.04 aarch64, 4 vCPU, 3.9 GB memory	n/a	Phase 0 report §3

Table 3. Software stack (versions pinned for reproducibility).

Layer	Component and version	Purpose
Model pipeline	Python 3.10; torch 2.5.1; transformers 4.46.3; onnx 1.17.0; onnxruntime 1.20.1; numpy 1.26.4	Split, export, quantize, train exit heads, validate
Browser runtime	onnxruntime-web 1.20.1 (WASM execution provider)	Runs each ONNX component
Package	`adaptive-onnx-loader` 0.1.0 (ES modules, TypeScript declarations); Node ≥ 18 for tests; dev dependency fake-indexeddb 6.0.0	Library and its unit tests
Browsers	Headless Chromium 153 (cloud, Playwright); Google Chrome 155 (Mac)	Experiments and demo
Server	`demo/server.mjs` (Node), sends COOP / COEP / CORP headers; `bench/throttledServer.mjs` for bandwidth limits	Cross-origin isolation; simulated links
Model and data	distilbert-base-uncased-finetuned-sst-2-english (Apache-2.0); SST-2 validation (872) and train parquet files	Model and evaluation data (`THIRD_PARTY_NOTICES.md`)

Sources: Phase 0 report §3; `package.json`; `packages/adaptive-onnx-loader/package.json`; `THIRD_PARTY_NOTICES.md`.

Table 4. Functional and non-functional requirements.

ID	Requirement	How it is verified
FR-1	Split the model into four components whose chained output equals the full model	Max |Δlogit| (Phase 1 V-checks)
FR-2	Run each component in its own Web Worker and free its memory by terminating the Worker	Memory-release proof (Phase 3, D-3)
FR-3	Cache components in IndexedDB, keyed and verified by SHA-256; never store a bad download	Unit tests; corruption test (Phase 4)
FR-4	Choose an execution plan per device profile; explain the choice; fail with a typed error if no plan is feasible	Table-driven unit tests; browser gates (Phase 5)
FR-5	Offer early exit and mixed precision, off by default, subject to the accuracy floor	Exit points equal the reference (Phases 5–6)
FR-6	Expose a small public API (`loadModel`, `AdaptiveModel`) and a demo that uses only that API	Installed-tarball browser test (Phase 6)
NFR-1	Correctness: int8 outputs bit-identical to the WASM reference; fp32 within 1e-4 of PyTorch	Every experiment row checks outputs
NFR-2	Determinism: same inputs give the same plan	Unit tests
NFR-3	Memory safety: predicted peak never below the measured peak	E-ENG (cloud), MAC-2 (Mac)
NFR-4	Reproducibility: pinned versions; raw data kept; analyses generated from raw files	Phase 7 raw-data index

2.11 Software development methodology

The project followed a **phase-gated incremental** life cycle. In an incremental model the system is delivered as a series of increments, each adding functionality to a working base, with feedback after each increment (Sommerville 2016). Here each phase delivered one

increment (for example, “components run in Workers” or “components are cached”) on top of the previous ones, and each phase passed through the same cycle defined in the Master Plan: *define → execute → validate → log issues → phase report → approval*. A phase could not start until the student had reviewed the previous phase report and approved it; this is the “gate” in Figure 4. No other process framework was used, and the project did not use sprints, daily meetings or other ceremonies of agile frameworks.

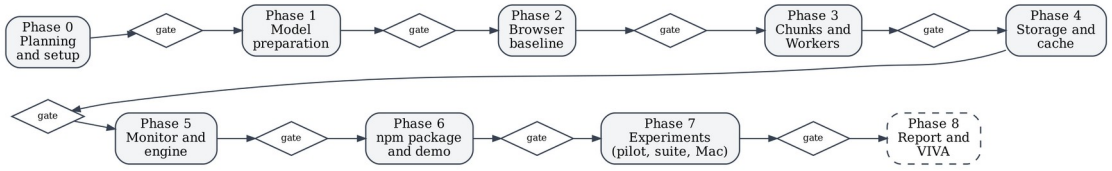


Figure 4. Phase-gated incremental life cycle: each phase ends at an approval gate (Master Plan §4).

Three practices supported the gates. Each phase report recorded validation checks with pass or fail, issues and deviations. A **findings register** (`docs/FINDINGS.md`) recorded every observation with its evidence; entries are never deleted, and later phases mark them as confirmed or superseded. Before the final experiments, the hypotheses, predictions and analysis rules were **pre-registered** in an experiment plan (`docs/PHASE_7_EXPERIMENT_PLAN.md`, v1.2), and a pilot was run to test the harness before the full suite. Table 5 lists the phases with their planned and actual dates. The schedule was planned over 30 days, but Phases 0–6 were completed on 31 October and Phase 7 on 1 November 2026, so the plan’s dates were not the binding constraint; the approval gates were.

Table 5. Phases, increments and approvals.

#	Phase (increment)	Planned	Actual	Gate outcome
0	Planning and set-up	31 Oct – 1 Nov	31 Oct	Approved 31 Oct
1	Model preparation (Python)	2 – 6 Nov	31 Oct	Approved 31 Oct; commit 437548f (with Phase 0)
2	Baseline browser inference	7 – 9 Nov	31 Oct	Approved 31 Oct after the Mac check; commit a34a391
3	Chunked execution and Workers	10 – 13 Nov	31 Oct	Approved 31 Oct; commit b5ba76e
4	Storage and cache layer	14 – 16 Nov	31 Oct	Approved 31 Oct; commit 8433ae0
5	Resource monitor and decision engine	17 – 20 Nov	31 Oct	Approved 31 Oct; commit 54945c3
6	npm package and demo	21 – 23 Nov	31 Oct	Approved 31 Oct; commit caae0e9
7	Experiments (pilot, suite, Mac)	24 – 27 Nov	31 Oct – 1 Nov	Approved 1 Nov; commit pending review
8	Report and VIVA preparation	28 – 30 Nov	in progress	—

Source: `PROGRESS_TRACKER.md` (updated 1 Nov 2026); `docs/00_MASTER_PLAN.md` §4; git log.

3 Practical Implementation

This chapter documents what was built, in the order in which it was built and validated. Sections 3.1 and 3.2 give an overview and the architecture. Sections 3.3 to 3.5 cover the offline model pipeline and the browser baseline. Sections 3.6 to 3.10 cover the four mechanisms that make loading adaptive: Worker-per-component execution, execution policies, the cache, the decision engine, and early exit with mixed precision. Section 3.11 describes the package and its public interface, and Section 3.12 the testing. Section 3.13 presents the experimental method and the results.

The following conventions are used, as in the sample report. File paths and identifiers are written in `monospace`. Source code is shown as numbered figures with the repository file and line range in the caption; the excerpts are copied verbatim from the repository at the time of writing, and an ellipsis (…) marks omitted lines. Measurements are labelled with their configuration: **CLOUD-HL-1T** is headless Chromium on the 2-vCPU cloud machine with one thread per session; **MAC-CHROME-*** is Chrome on the Mac. Screenshots were captured for documentation from the unchanged demo pages; the numbers visible in them are illustrations, not measurements of record.

3.1 System overview

The system lets a web page classify sentences with DistilBERT-SST2 while controlling how much of the model is in memory. Its main features are:

- **Lossless split** of the model into four ONNX components, each available in fp32 and int8, plus two early-exit heads.
- **Worker-per-component execution:** each loaded component lives in its own Web Worker; terminating the Worker destroys its WebAssembly heap (measured in the cloud, Section 3.6).
- **Three execution shapes:** resident (all components stay loaded), sequential (one component at a time) and hybrid (some components pinned), each with a group size and an optional pre-start of the next component.
- **Persistent cache** in IndexedDB, keyed by SHA-256 and verified on every read.
- **Decision engine** that chooses a plan for a device profile under hard memory and accuracy constraints and explains the choice in six numbered sentences.
- **Optional early exit and mixed precision**, both off by default.
- **Public API and demo:** `loadModel()` returns an `AdaptiveModel` with `infer()`, `explainPlan()`, `previewPlan()`, `getCacheStats()`, `replan()`, `clearCache()` and `dispose()`.

Figure 5 shows what happens when a page uses the library, from `loadModel()` to the first result.

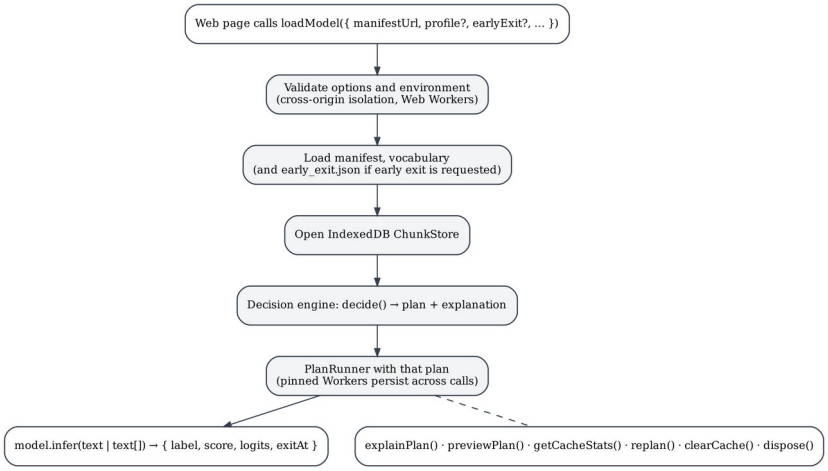


Figure 5. Flow of a `loadModel()` call and later `infer()` calls (from `src/api.js` and Phase 6 report §2).

3.2 System architecture

The system has an offline part and a browser part (Figure 6). The offline part is a Python pipeline that turns the Hugging Face checkpoint into ONNX components and a **manifest**, a JSON file that is the contract between the build and the runtime: it lists every component and precision with its file, size in bytes, SHA-256 hash and input/output signature, together with the chain order, the exit points and the tokenizer settings. The browser part is the JavaScript package. Its modules on the page side tokenize text, run the decision engine and drive a `PlanRunner`; on the Worker side, each `chunkWorker.js` instance obtains its component’s bytes through the cache layer and runs it with ONNX Runtime Web. Table 6 summarises the modules.

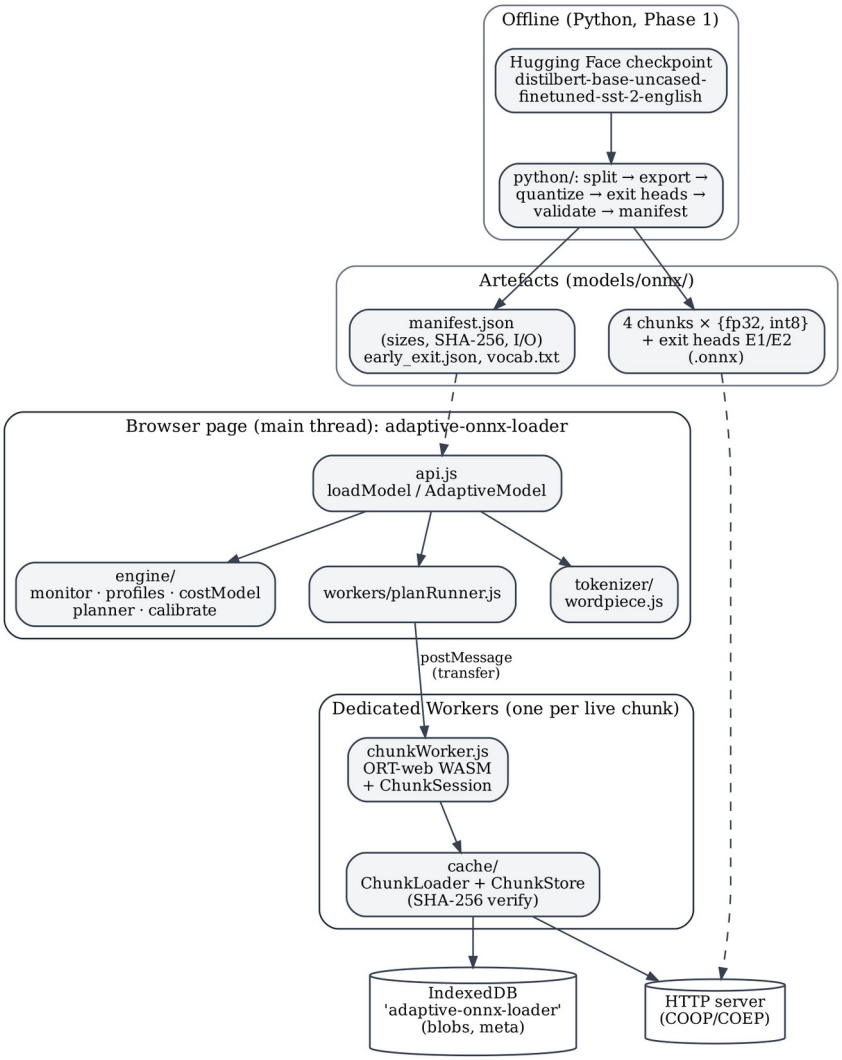


Figure 6. System architecture: offline pipeline, artefacts, page-side package modules and per-component Workers.

Table 6. Package modules and their responsibilities (`packages/adaptive-onnx-loader/src/`).

Module	Responsibility	Phase
`tokenizer/wordpiece.js`	WordPiece tokenizer equivalent to the Python tokenizer (uncased, 30,522-token vocabulary)	2
`baseline/fullModel.js`, `metrics/memory.js`	Unsplit baseline; WebAssembly-heap probe and browser memory measurement	2
`manifest.js`, `chunks/chain.js`, `chunks/chunkSession.js`, `tensors.js`	Manifest access; chaining rule; one `InferenceSession` per component; transferable tensors	3
`workers/chunkWorker.js`, `workerChunk.js`, `workerChainRunner.js`	Worker hosting one component; page-side handle; resident / sequential runners	3
`cache/chunkStore.js`, `chunkLoader.js`, `hash.js`, `memoryLRU.js`	IndexedDB store, tiered loader with SHA-256 verification, in-memory LRU	4

Module	Responsibility	Phase
`workers/workerPool.js`, `pooledChainRunner.js`	Pool of live Workers under a heap budget (hot tier)	4
`engine/monitor.js`, `profiles.js`, `measurements.js`, `costModel.js`, `planner.js`, `calibrate.js`	Signals and profiles; measured inputs; memory and time model; 7-step decision; per-device timing calibration	5
`workers/planRunner.js`	Executes any plan: precision, pinned set, group size, pre-start, early exit	5
`api.js`, `index.js`, `index.d.ts`	Public API, exports and TypeScript declarations	6

3.3 Model preparation and splitting

The split is defined in `python/adaptive_split/chunks.py` (Figure 7). Each component is a small PyTorch module that **wraps** (does not copy) the corresponding sub-modules of the loaded `DistilBertForSequenceClassification`, and whose inputs and outputs are plain tensors so that it can be exported as a standalone ONNX graph. The tensor crossing every boundary is the hidden state, float32 [batch, seq, 768]; the attention mask, int64 [batch, seq], is passed to every layer component because each attention layer needs it to ignore padding.

```python
# name -> which transformer layers it owns (None = not a layer group)
CHUNK_SPEC = {
    "c0_embed":  {"layers": None,   "kind": "embed"},
    "c1_l0_1":   {"layers": (0, 2), "kind": "layers"},
    "c2_l2_3":   {"layers": (2, 4), "kind": "layers"},
    "c3_l4_5_cls": {"layers": (4, 6), "kind": "classifier"},
}
# early-exit heads attach after these chunks
EXIT_POINTS = {"e1_after_c1": "c1_l0_1", "e2_after_c2": "c2_l2_3"}


class EmbeddingChunk(nn.Module):
    """C0: word + position embeddings + LayerNorm.  input_ids[B,S] -> hidden[B,S,768]"""
    def __init__(self, embeddings: nn.Module):
        super().__init__()
        self.embeddings = embeddings

    def forward(self, input_ids: torch.Tensor) -> torch.Tensor:
        return self.embeddings(input_ids)


class LayerGroupChunk(nn.Module):
    """C1/C2: a contiguous slice of TransformerBlocks. (hidden, mask) -> hidden"""
    def __init__(self, blocks: list[nn.Module]):
        super().__init__()
        self.blocks = nn.ModuleList(blocks)

    def forward(self, hidden: torch.Tensor, attention_mask: torch.Tensor) -> torch.Tensor:
        for blk in self.blocks:
            # eager attention expects a 2-D [B,S] mask and expands it internally
            hidden = blk(x=hidden, attn_mask=attention_mask)[-1]
        return hidden
```

Figure 7. Component definitions: the split specification and the layer-group module.

Source: `python/adaptive_split/chunks.py`, lines 17–48.

`split_model.py` verifies the split before anything is exported. It runs all 872 validation sentences through the original model and through the chain of components; the maximum logit difference was **0.0** (bit-identical) with 100% agreement. It also checks that the parameter counts of the components (23,835,648 + 14,175,744 + 14,175,744 + 14,767,874) add up exactly to the model total of 66,955,010, so no weight is missing or duplicated (Phase 1 report §3, Step 1; F-1). Table 7 lists the resulting files.

Table 7. Model components and file sizes (Phase 1 report §2, “Artifacts”).

Component	Contents	Inputs → outputs	fp32	int8
`c0_embed`	Word + position embeddings, LayerNorm	`input_ids` → `hidden`	95.3 MB	23.8 MB
`c1_l0_1`	Transformer layers 0–1	`hidden_in`, `attention_mask` → `hidden`	56.7 MB	14.3 MB

Component	Contents	Inputs → outputs	fp32	int8
`c2_l2_3`	Transformer layers 2–3	`hidden_in`, `attention_mask` → `hidden`	56.7 MB	14.3 MB
`c3_l4_5_cls`	Layers 4–5, pre-classifier, classifier	`hidden_in`, `attention_mask` → `logits`	59.1 MB	14.9 MB
`e1_after_c1`, `e2_after_c2`	Exit heads on the [CLS] vector after layer 1 / layer 3	`hidden_in` → `logits`	2.4 MB each	0.6 MB each
`full`	Unsplit baseline	`input_ids`, `attention_mask` → `logits`	267.9 MB	67.4 MB

The components add up to the full model with no byte overhead: 267.9 vs 267.9 MB (fp32), 67.3 vs 67.4 MB (int8). The browser uses the four main components in fp32 and int8 and the exit heads in fp32 (`workers/planRunner.js`, line 27).

3.4 ONNX export, quantization and validation

The whole offline pipeline is reproduced by one script, `python/run_phase1.sh`, whose steps are shown in Figure 8. Each step is idempotent, and the slow feature-extraction step is sharded and resumable.

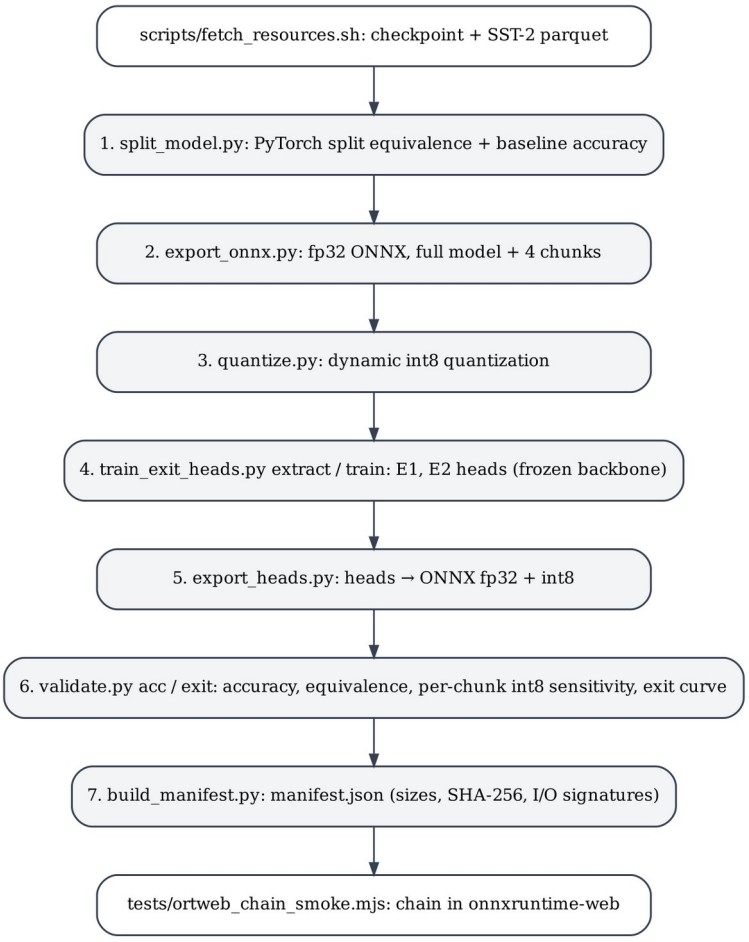


Figure 8. Offline model pipeline (`python/run_phase1.sh`).

Export. `export_onnx.py` exports the full model and each component with `torch.onnx.export` at opset 17, with dynamic batch and sequence dimensions. Each

exported graph contains only its own component's weights. **Quantization.** `quantize.py` applies ONNX Runtime dynamic quantization to every graph (Figure 9). Only `MatMul` and `Gather` operators are quantized, and `MatMulConstBOnly` restricts quantization to products with a constant weight matrix, so the attention-score products, which multiply two activations, stay in fp32 (Phase 1 report §3, Step 3). Quantizing `Gather` also shrinks the embedding table (30,522 × 768), the largest single tensor.

```
21 def main():
22     DST.mkdir(parents=True, exist_ok=True)
23     for f in sorted(SRC.glob("*.onnx")):
24         SCRATCH.mkdir(parents=True, exist_ok=True)
25         src, tmp_out, out = SCRATCH / f.name, SCRATCH / ("q_" + f.name), DST /
f.name
26         shutil.copy(f, src)
27         quantize_dynamic(str(src), str(tmp_out), weight_type=QuantType.QInt8,
28                          op_types_to_quantize=["MatMul", "Gather"],
29                          extra_options={"MatMulConstBOnly": True})
```

Figure 9. Dynamic int8 quantization of each exported graph.

Source: `python/quantize.py`, lines 21–29.

Exit heads. `train_exit_heads.py` trains two heads with the backbone frozen, so the main path keeps exactly its 91.06%. Each head has the shape of the final classifier (Linear 768→768, ReLU, Linear 768→2) and is initialised from it. Features were extracted for 20,000 training sentences (seed 0, maximum length 64); the heads were trained with AdamW (learning rate 1e-4, 8 epochs, batch 128) and the best epoch was selected on a 2,000-sentence development split. On the validation set E1 reached 76.4% and E2 81.3% (Phase 1 report §3, Step 4).

Validation. `validate.py` runs all 872 validation sentences through ONNX Runtime and compares every configuration with the PyTorch logits (Table 8). The fp32 chain matched PyTorch to a maximum logit difference of 2.4×10^{-5} with 100% agreement. The per-component sensitivity test, in which only one component is int8, identified C2 as the most sensitive component (20 flipped predictions) and C0 as the least (4). Finally, `build_manifest.py` writes the manifest (Figure 10), and `tests/ortweb_chain_smoke.mjs` runs the chain in ONNX Runtime Web under Node, confirming that the WASM back-end supports the int8 operators before any browser work started.

Table 8. Offline validation in native ONNX Runtime, batch 32 (Phase 1 report §3, Step 6).

Configuration	Accuracy	Max |Δlogit| vs PyTorch	Agreement
full fp32 / chain fp32	91.06% / 91.06%	2.4e-5 / 2.4e-5	100% / 100%
full int8 / chain int8	90.37% / 90.37%	2.25 / 2.25	98.39% (identical)

Configuration	Accuracy	Max |Δlogit| vs PyTorch	Agreement
Only C0 int8 / only C1 / only C2 / only C3	90.60% / 90.60% / 90.60% / 90.94%	—	4 / 14 / 20 / 5 flips

These are native (CPU) ONNX Runtime results with batch 32 on the Mac's Linux sandbox. The browser uses WebAssembly kernels at batch 1, where int8 accuracy is 90.02%; the difference is explained in Section 3.5 and measured in Section 3.13 (E-B, F-5).

```
226      "c2_l2_3": {
227        "layers": [
228          2,
229          4
230        ],
231        "fp32": {
232          "file": "fp32/c2_l2_3.onnx",
233          "bytes": 56727109,
234          "sha256":
"9362004377eb5727ec9f05b85729b91d7f52a42caa51b7a15b424234be18fa1b",
     …
266        "int8": {
267          "file": "int8/c2_l2_3.onnx",
268          "bytes": 14282351,
269          "sha256":
"0068dcc8b0148f2ffd70f5bccf27d1782e41020c9d11d7a405bc8bd640a07d65",
270          "inputs": [
```

Figure 10. Manifest entry for component C2 (fp32 part shown first, then the start of the int8 part).

Source: `models/onnx/manifest.json`, lines 226–234, 266–270.

3.5 Browser baseline inference

Before splitting anything in the browser, the unsplit model was run in ONNX Runtime Web to establish a correct baseline and a trustworthy memory metric (Phase 2). Three parts were needed.

Tokenizer. The model only understands token IDs, so the browser must produce exactly the IDs that the Python tokenizer produced. `tokenizer/wordpiece.js` re-implements the uncased BERT WordPiece tokenizer without dependencies: text cleaning, spaces around CJK characters, accent stripping, lower-casing, punctuation splitting, greedy longest-match WordPiece against the 30,522-token vocabulary, special tokens, truncation to 128 tokens, padding and the attention mask. It was tested against Python on 26 hand-picked edge cases and on all 872 validation sentences (Phase 2 report §3.1).

Server. `demo/server.mjs` serves the project with the COOP and COEP headers, so pages are cross-origin isolated and ORT can use threads and the browser's memory API (Section 2.5).

Memory probe. The browser's whole-page memory API, `performance.measureUserAgentSpecificMemory()` (MDN Web Docs 2026j), turned out to be impractical as the main metric: it waited about 16 s for garbage collection and took

about 60 s per call once ORT Worker threads existed (F-7). The project therefore measures the WebAssembly heap directly (Figure 11). The probe replaces the `WebAssembly.Memory` constructor before ORT starts, keeps a reference to every memory created, and reports the sum of their sizes. In the cloud it agreed with the browser's whole-page measurement to within about 5 MiB (F-8), and that API was kept for spot checks.

```
17  /** Must run BEFORE onnxruntime-web initialises its WASM module. Idempotent. */
18  export function installWasmMemoryProbe() {
19    if (globalThis.__wasmMemoryProbe) return;
20    const Native = WebAssembly.Memory;
21    function ProbedMemory(desc) { const m = new Native(desc); probed.push(m); return
m; }
22    ProbedMemory.prototype = Native.prototype;
23    WebAssembly.Memory = ProbedMemory;
24    globalThis.__wasmMemoryProbe = true;
25  }
26
27  export function wasmHeapBytes() {
28    return probed.reduce((s, m) => s + m.buffer.byteLength, 0);
29  }
```

Figure 11. WebAssembly-heap probe used as the primary memory metric.

Source: `packages/adaptive-onnx-loader/src/metrics/memory.js`, lines 17–29.

Table 9 shows the baseline in the cloud. Two results shaped the rest of the project. The heap was about **2.6–2.8 times the file size**, because the bytes exist in several forms during loading (the fetched buffer, a copy inside WebAssembly, and the parsed weights); so peak memory, not file size, is the constraint. And after `session.release()` the heap **stayed at 656 / 180 MiB**. The same retention was measured on the Mac (MAC-CHROME-4T: int8 181.5 → 181.5 MiB, fp32 696.9 → 696.9 MiB; F-4). Figure 12 shows the baseline page.

Table 9. Browser baseline, unsplit model (CLOUD-HL-1T for 1 thread, CLOUD-HL-2T for 2 threads; 872 sentences, batch 1; Phase 2 report §2).

Precision, threads	Accuracy	Download	Load (fetch + session)	p50 latency	WASM heap
fp32, 1	91.06%	267.9 MB	1,569 ms	70.2 ms	656 MiB
fp32, 2	91.06%	267.9 MB	1,793 ms	37.1 ms	657 MiB
int8, 1	90.02%	67.4 MB	752 ms	43.1 ms	180 MiB
int8, 2	90.02%	67.4 MB	948 ms	27.6 ms	181 MiB

Load times on the shared cloud machine varied widely between runs (F-9), so later experiments use five repetitions and report medians.

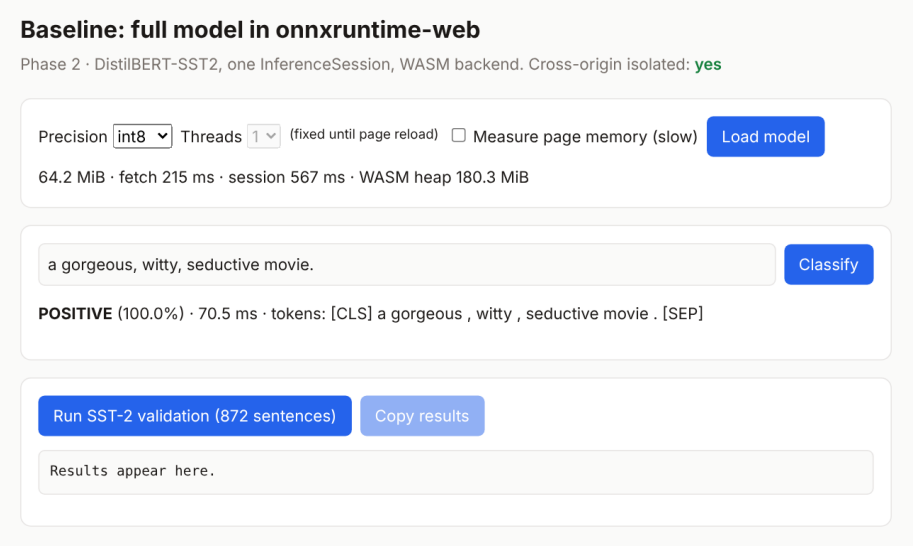


Figure 12. Baseline page (`demo/phase2-baseline.html`) after loading the int8 model and classifying one sentence.

Documentation capture, CLOUD-HL, 1 thread.

The int8 reference. The first browser run failed one check: browser int8 accuracy was 90.02%, while native Python ONNX Runtime at batch 1 gave 90.14%, a difference of exactly one sentence. Investigation (`python/diag_int8_parity.py`) showed that graph optimisation was not the cause; the native CPU and WebAssembly int8 kernels round differently, and the WebAssembly results were bit-identical across x86 Chromium and ARM Node. With approval (D-2.1), the int8 correctness criterion was changed to "bit-identical to the WebAssembly reference", and the native difference was recorded as a finding to be reported (F-5). The same investigation found that native int8 accuracy depends on the batch size (90.37% at batch 32 vs 90.14% at batch 1), which is why the runtime always uses batch 1 (F-6; Phase 2 report §5).

3.6 Worker-based component execution

Phase 3 moved each component into its own dedicated Worker. A `ChunkSession` wraps one `InferenceSession`; `chunks/chain.js` defines the chaining rule (C0 takes `input_ids`; C1–C3 take `hidden_in` and `attention_mask`; the previous component's `hidden` becomes the next component's `hidden_in`); and `tensors.js` converts ORT tensors into plain objects whose buffers can be transferred. The Worker script `workers/chunkWorker.js` hosts exactly one component and answers four request types, matched by an `id` (Figure 13 and Figure 14).

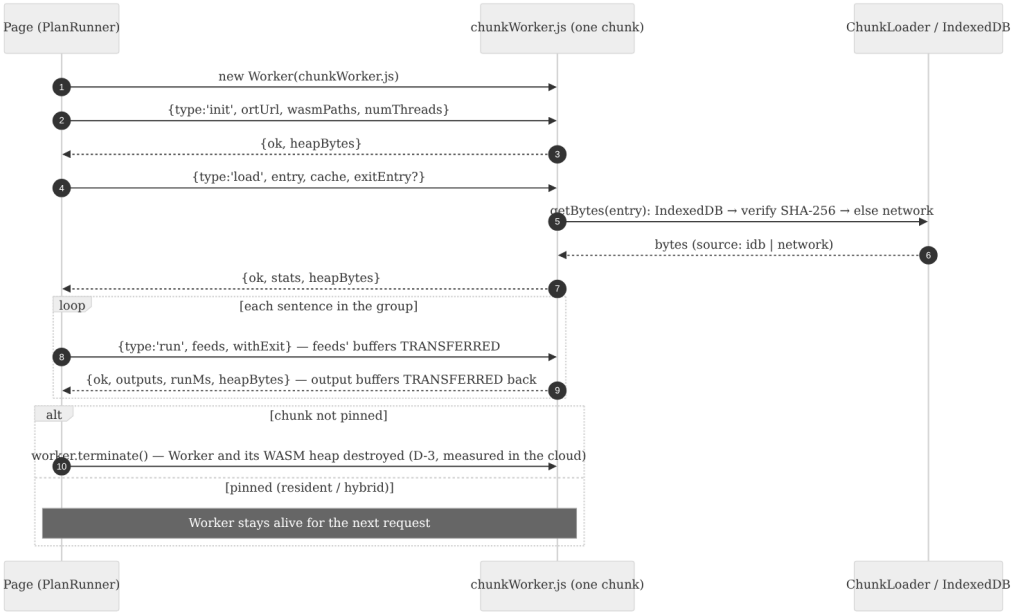


Figure 13. Message sequence between the page and one component Worker (protocol from the header of `workers/chunkWorker.js`).

```
  8   * Protocol (request/response, matched by `id`):
  9   *   → { id, type:'init', ortUrl, wasmPaths, numThreads }      ← { id, ok,
heapBytes }
 10   *   → { id, type:'load', entry, sessionOptions, cache? }       ← { id, ok, stats,
heapBytes }
 11   *        cache (optional, Phase 4): { dbName, budgetBytes, verifyOnRead } — the
Worker gets the
 12   *        chunk's bytes from IndexedDB itself (or downloads, verifies and stores
them), so bytes
 13   *        never pass through the page. Without `cache` the behaviour is exactly
Phase 3's.
 14   *   → { id, type:'run',  feeds, withExit? }  (feeds' buffers TRANSFERRED) ← { id,
ok, outputs, runMs, heapBytes } (outputs transferred back)
 15   *        Phase 5: if `load` also received `exitEntry`, the exit head lives in
THIS Worker (same WASM
 16   *        instance, no extra Worker overhead) and `withExit: true` adds
`outputs.exitLogits`, computed
 17   *        from this chunk's output hidden state.
 18   *   → { id, type:'release' }                                   ← { id, ok,
heapBytes }
 19   * Errors: ← { id, ok:false, error }
      …
 56     async run({ feeds, withExit = false }) {
 57       const t0 = performance.now();
 58       const outputs = await chunk.run(feeds);
 59       if (withExit && exitHead) outputs.exitLogits = (await
exitHead.run({ hidden_in: outputs.hidden })).logits;
 60       return { outputs, runMs: performance.now() - t0, _transfer:
transferList(outputs) };
 61     },
 62     async release() { if (chunk) await chunk.release(); if (exitHead) await
exitHead.release(); chunk = exitHead = null; return {}; },
 63   };
 64
 65   self.onmessage = async ({ data: msg }) => {
 66     try {
 67       const { _transfer = [], ...res } = await handlers[msg.type](msg);
 68       self.postMessage({ id: msg.id, ok: true, heapBytes: wasmHeapBytes(), ...res },
_transfer);
 69     } catch (e) {
 70       self.postMessage({ id: msg.id, ok: false, error: String(e?.stack || e),
heapBytes: wasmHeapBytes() });
 71     }
 72   };
```

Figure 14. Worker protocol and message handling; output buffers are returned in the transfer list.

Source: `packages/adaptive-onnx-loader/src/workers/chunkWorker.js`, lines 8–19, 56–72.

Two details of Figure 14 matter. Every reply carries the Worker's current heap size (`heapBytes`), which is how the page tracks live memory without the slow browser API. And `run` returns the output tensors together with a transfer list, so the hidden state moves to the page without copying; the input feeds are transferred to the Worker in the same way. Because import maps do not apply inside Workers, the URL of ONNX Runtime is passed in the `init` message.

The decisive measurement of Phase 3 is the **memory-release proof** (Table 10). It was taken with the browser's own memory API, independently of the probe. Calling `session.release()` in every Worker freed nothing; terminating the Workers returned the page's measured memory to its 2.8 MiB starting point (D-3). This supported the design choice: in this measurement, terminating a Worker released the memory that releasing its session did not. The scope of the evidence is limited: it is one measurement per precision, in the cloud browser only (it was not repeated on the Mac), and it reports the memory the browser attributes to the page, not memory returned to the operating system.

Table 10. Memory-release proof (CLOUD-HL-1T, `measureUserAgentSpecificMemory`; Phase 3 report §2).

Step	int8 page memory	fp32 page memory
0 · page only	2.8 MiB	2.8 MiB
1 · four component Workers loaded	223.4 MiB	822.7 MiB
2 · `session.release()` in every Worker	223.5 MiB (0% freed)	822.7 MiB (0% freed)
3 · `worker.terminate()` on all four	2.8 MiB (100% freed)	2.8 MiB (100% freed)

Passing tensors between Workers was cheap. In resident mode the messaging overhead was 1.17 ms of 44.2 ms per sentence for int8 (2.6%) and 1.34 ms of 71.2 ms for fp32 (1.8%), and outputs stayed identical to the baseline (Phase 3 report §5.2). The cost of isolation is a fixed overhead per Worker: four resident Workers needed 214.1 MiB (int8) and 813.3 MiB (fp32), more than the single-session baseline (180 / 656 MiB), because each Worker has its own runtime instance (Phase 3 report §2). Figure 15 shows the Phase 3 test page after a resident run.

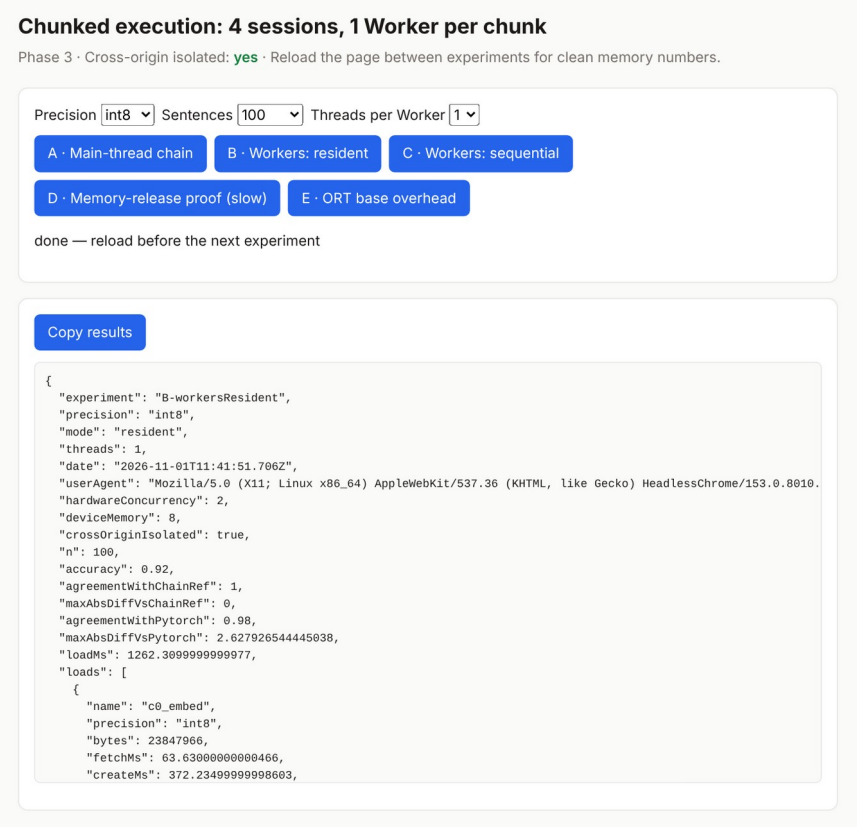


Figure 15. Phase 3 test page (`demo/phase3-chunks.html`) after experiment B, resident Workers, int8, 100 sentences.

The result records the browser-reported `hardwareConcurrency` 2 and `deviceMemory` 8 of the cloud browser. Documentation capture, CLOUD-HL.

3.7 Execution policies

A **plan** fixes five things: the precision plan (fp32, int8 or mixedC2), the set of **pinned** components whose Workers stay alive between requests, the **group size** G (how many sentences pass through a component before a non-pinned component is unloaded), whether to **pre-start** the next non-pinned component while the current one computes, and whether early exit is on. The pinned set determines the shape: all four pinned is **resident**, none pinned is **sequential**, and anything in between is **hybrid**. Figure 16 illustrates the shapes schematically.

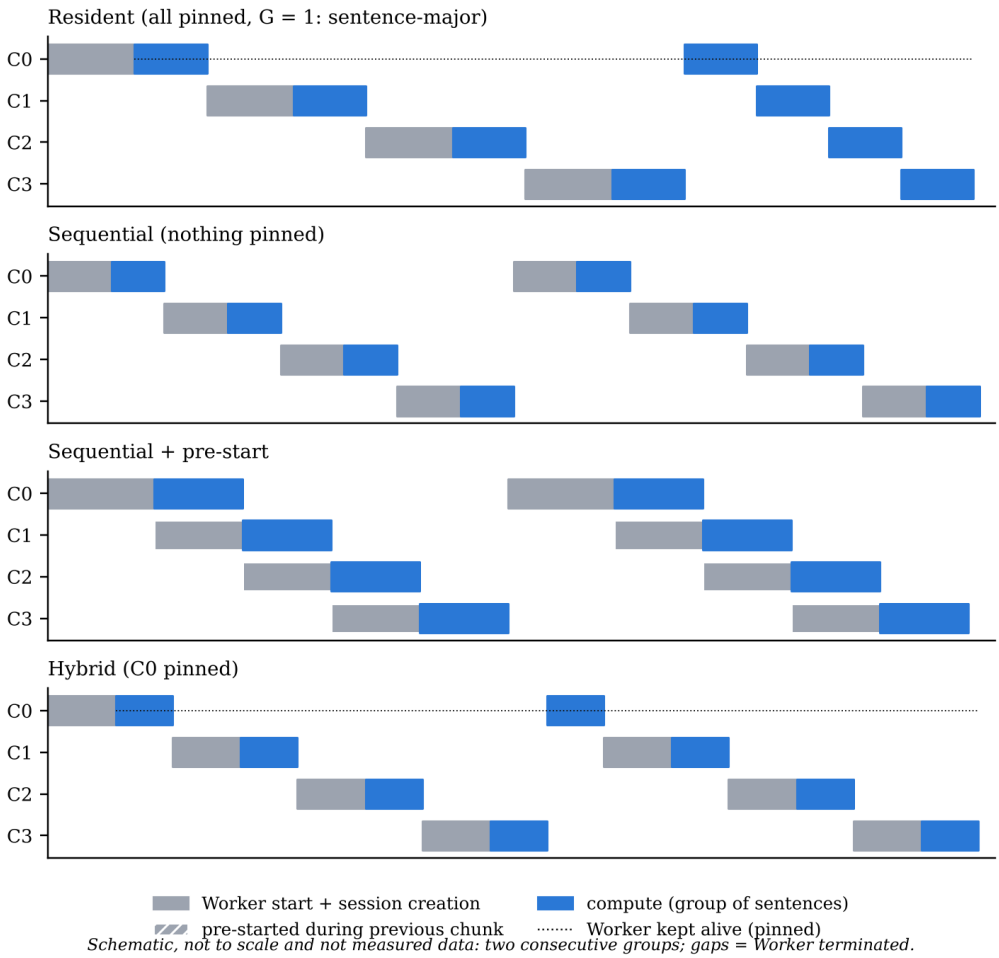


Figure 16. Which component Workers are alive over time in each plan shape (schematic, not measured; semantics of `PlanRunner.classifyMany`).

All shapes are executed by one class, `PlanRunner` (Figure 17). For each group of sentences it walks the chain: it reuses a live Worker or spawns one (or awaits the one being pre-started), optionally pre-starts the next non-pinned component, runs every still-active sentence through the component, applies early exit, and terminates the component unless it is pinned. Every sentence still runs at batch size 1, which keeps int8 results identical to the reference (F-6). In sequential plans the page holds the hidden states of the sentences waiting for the next component; with 872 sentences in one group this reached 64.3 MiB (Phase 3 report §5.4; F-21), which is why the group size became a plan dimension with values 1, 8 and 32.

```
54        for (let i = 0; i < this.entries.length && active.length; i++) {
55          const e = this.entries[i];
56          let w = this.live.get(e.name);
57          if (!w) w = warming?.name === e.name ? await warming.p : await
this.spawn(e);
58          warming = null;
59          const next = this.entries[i + 1];
60          if (this.plan.prewarm && next && !this.pinned.has(next.name) && !
this.live.has(next.name))
61            warming = { name: next.name, p: this.spawn(next) };            //
overlaps with the runs below
62          const withExit = this.tau != null && !!EXIT_AFTER[e.name];
63          const still = [];
64          for (const j of active) {
65            const s = st.get(j), f = feedsFor(e.name, i, s);
66            if (f.attention_mask) f.attention_mask = cloneT(f.attention_mask);
67            const { outputs, runMs, overheadMs } = await w.run(f, { withExit });
68            const tr = trace.get(j); tr.path.push(e.name); tr.computeMs += runMs;
tr.transferMs += overheadMs;
69            if (outputs.logits) { results[j] = { logits: logitsRows(outputs.logits)
[0], exitAt: 'full', ...tr }; continue; }
70            s.hidden = outputs.hidden; if (i === 0) delete s.input_ids;
71            if (withExit) {
72              const lg = logitsRows(outputs.exitLogits)[0];
73              if (conf(lg) >= this.tau) { tr.path.push(EXIT_AFTER[e.name]);
results[j] = { logits: lg, exitAt: e.name === 'c1_l0_1' ? 'e1' : 'e2', ...tr };
continue; }
74            }
75            still.push(j);
76          }
77          active = still;
78          if (!this.pinned.has(e.name)) this.kill(e.name);
79          if (!active.length && warming) { await warming.p; this.kill(warming.name);
warming = null; }   // nothing left to compute
80        }
```

Figure 17. Core loop of `PlanRunner`: spawn or reuse, pre-start, run, early exit, terminate unless pinned.

Source: `packages/adaptive-onnx-loader/src/workers/planRunner.js`, lines 54–80.

The memory cost of each shape is predictable from the per-component heaps. Peak Worker memory is the sum of the pinned heaps plus the largest transient heap among the non-pinned components; with pre-start, a non-pinned component and the next one can be alive at the same time (Phase 5 report §5.3). Table 11 shows the measured peaks from Phase 3; the controlled Phase 7 measurements are in Section 3.13.

Table 11. Peak WebAssembly memory by execution shape (CLOUD-HL-1T, 872 sentences; Phase 3 report §2).

Shape	fp32	int8
Baseline (one session)	656 MiB	180.3 MiB
Chain of four sessions on the main thread	403.4 MiB (−38%)	111.2 MiB (−38%)
Workers, resident	813.3 MiB (+24%)	214.1 MiB (+19%)

Shape	fp32	int8
Workers, sequential	280.1 MiB (−57%)	77.2 MiB (−57%)

Sequential figures exclude the hidden states waiting in the page; including them for all 872 sentences gives 344 MiB (fp32) and 141.5 MiB (int8).

3.8 IndexedDB caching and memory management

Without a cache, a sequential plan downloads every component on every request. Phase 4 added a tiered loader (Figure 18). T0 is a live Worker that already holds the component; T1 is the component's bytes in an in-memory LRU; T2 is IndexedDB; T3 is the network. The manifest's SHA-256 hash is the source of truth at every tier: downloaded bytes are verified before they are stored, so a bad download is never cached, and stored bytes are verified on every read, so a corrupted entry is deleted and fetched again (Figure 19).

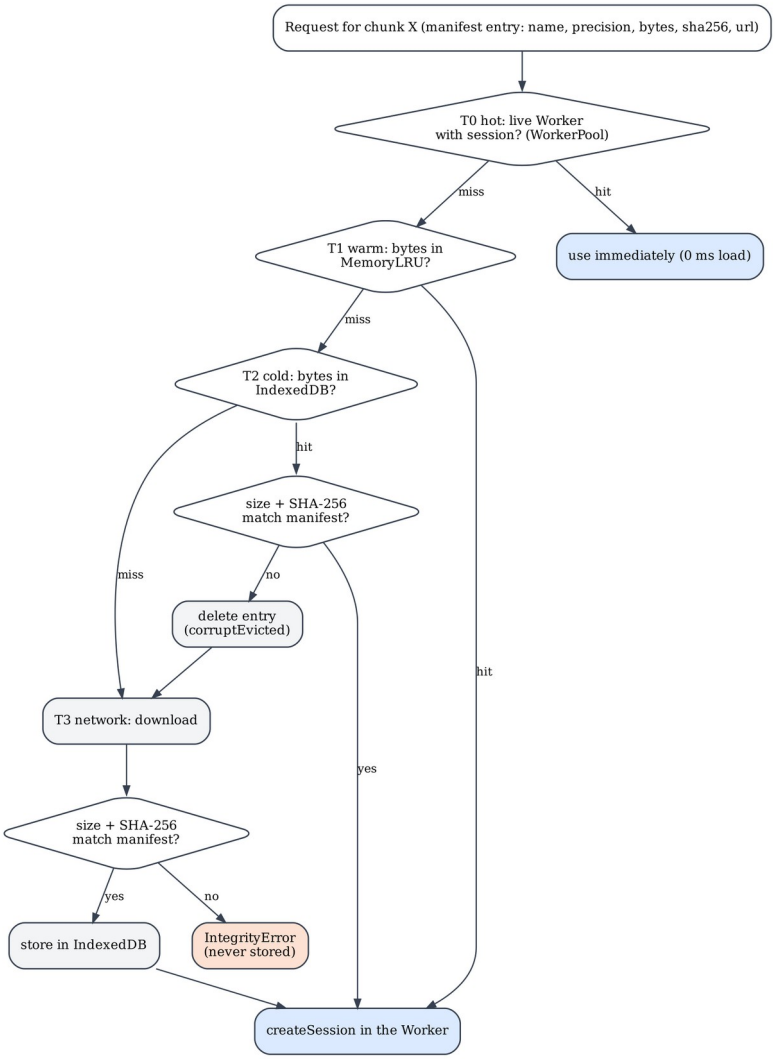


Figure 18. Tiered component loading with integrity checks (`cache/chunkLoader.js`, `workers/workerPool.js`; Phase 4 report §4.1).

```
35      if (this.store) {
36        const t0 = performance.now(); const hit = await
this.store.get(entry.sha256); t.idbReadMs = performance.now() - t0;
37        if (hit) {
38          const ok = this.verifyOnRead ? await this.verify(hit.bytes, entry, t) :
hit.bytes.byteLength === entry.bytes;
39          if (ok) { this.stats.idb++; this.memory?.set(entry.sha256, hit.bytes);
return { bytes: hit.bytes, source: 'idb', timings: t, corruptEvicted }; }
40          await this.store.delete(entry.sha256); corruptEvicted = true;
this.stats.corruptEvicted++;
41        }
42      }
43      const t1 = performance.now();
44      const res = await this.fetchImpl(entry.url);
45      if (!res.ok) throw new Error(`fetch ${entry.url}: HTTP ${res.status}`);
46      const bytes = new Uint8Array(await res.arrayBuffer()); t.fetchMs =
performance.now() - t1;
47      if (!(await this.verify(bytes, entry, t)))
48        throw new IntegrityError(`${entry.name}/${entry.precision}: downloaded bytes
do not match the manifest SHA-256`);
49      if (this.store) { const t2 = performance.now(); t.stored = await
this.store.put(entry.sha256, bytes, entry); t.idbPutMs = performance.now() - t2; }
50      this.memory?.set(entry.sha256, bytes);
51      this.stats.network++;
```

Figure 19. IndexedDB read with verify-on-read, fall-through to the network, verify-before-store.

Source: `packages/adaptive-onnx-loader/src/cache/chunkLoader.js`, lines 35–51.

The store (`cache/chunkStore.js`) uses two object stores so that the frequent bookkeeping updates never rewrite the large values (Figure 20): `blobs` maps a SHA-256 key to the component's bytes, and `meta` keeps name, precision, size, timestamps and hit count, with an index on last use. Because the key is the hash of the content, a new model version automatically gets new keys. When a cache is used, the Worker obtains the bytes itself, so they never pass through the page (Phase 4 report §4.4).

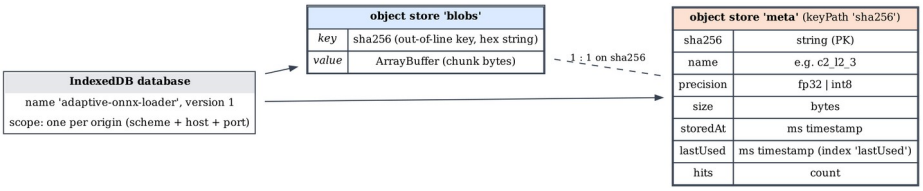


Figure 20. IndexedDB schema of the component store (from `ChunkStore.open()` in `cache/chunkStore.js`).

Verification is affordable: hashing ran at 0.4–1.1 GB/s in the cloud, 13–39 ms per int8 component and 63–115 ms per fp32 component, about 5–20% of a load, so verify-on-read stays on by default (F-25). On the Mac it cost 3.7% (int8) and 10% (fp32) of a warm request (Phase 4 report §13.3). A deliberately tampered component (same size, one byte flipped) was detected, deleted, fetched again and repaired, while the other three were still served from the cache (Phase 4 report §2).

Two findings from Phase 4 changed the design of Phase 5. First, an **LRU policy failed completely** when the budget was smaller than the model: components are always used in the same order C0 → C3, so with a cyclic access pattern each new component evicts the one that will be needed next, and the hit rate was 0% on every visit, both in the disk store and in the Worker pool (F-24; on the Mac, the pool under budget was 14.5–16.5 times slower per sentence). Pinning a subset of components, chosen by the engine, replaced LRU eviction for this purpose (issue I-4.1). Second, on a warm request most of the time was spent starting Workers and creating sessions, not reading bytes (F-22), which led to the resident, pinned and pre-start plans.

Figure 21 shows the cache test page after a cold and a warm request, and Figure 22 shows the contents of the store. As agreed for this report, Figure 22 is **an application-generated view, not a browser DevTools screenshot**: a documentation page (`docs/final-report/tools/idb_view.html`) reads the store with the package's own read-only calls and lists the `meta` entries.

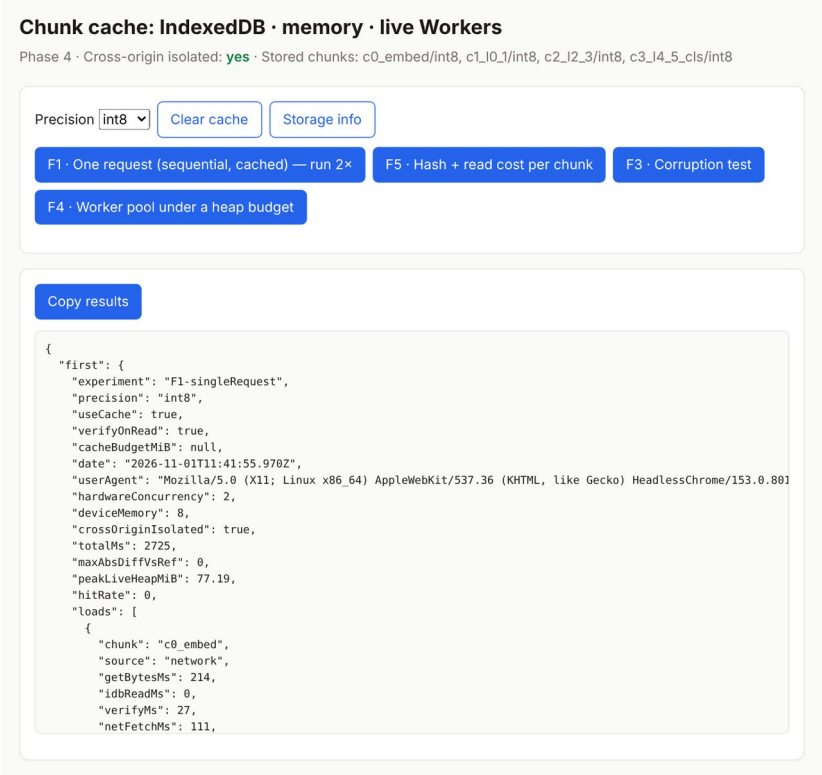
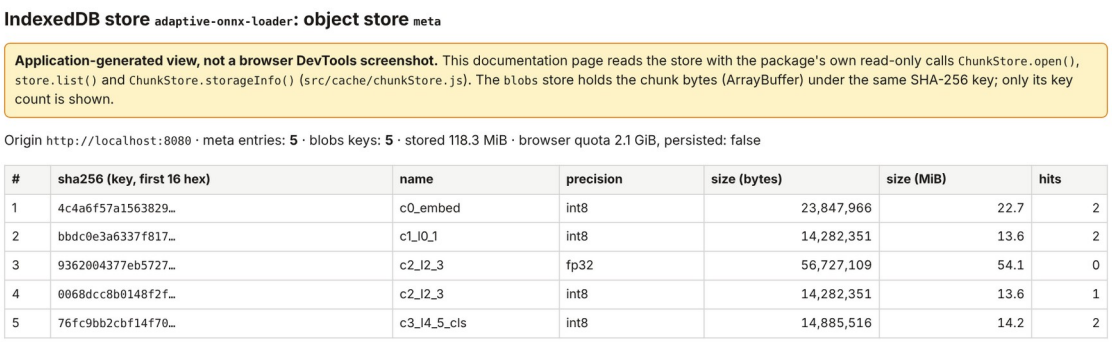


Figure 21. Cache test page (`demo/phase4-cache.html`) after experiment F1, one request run twice: the first fills the cache from the network, the second reads IndexedDB. Documentation capture, CLOUD-HL, localhost.



IndexedDB store `adaptive-onnx-loader`**: object store** `meta`

Application-generated view, not a browser DevTools screenshot. This documentation page reads the store with the package's own read-only calls `ChunkStore.open()`, `store.list()` and `ChunkStore.storageInfo()` (`src/cache/chunkStore.js`). The `blobs` store holds the chunk bytes (ArrayBuffer) under the same SHA-256 key; only its key count is shown.

Origin `http://localhost:8080` · meta entries: **5** · blobs keys: **5** · stored 118.3 MiB · browser quota 2.1 GiB, persisted: false

#	sha256 (key, first 16 hex)	name	precision	size (bytes)	size (MiB)	hits
1	4c4a6f57a1563829…	c0_embed	int8	23,847,966	22.7	2
2	bbdc0e3a6337f817…	c1_l0_1	int8	14,282,351	13.6	2
3	9362004377eb5727…	c2_l2_3	fp32	56,727,109	54.1	0
4	0068dcc8b0148f2f…	c2_l2_3	int8	14,282,351	13.6	1
5	76fc9bb2cbf14f70…	c3_l4_5_cls	int8	14,885,516	14.2	2

Figure 22. Application-generated view of the IndexedDB store after the documentation captures (five entries: the four int8 components and C2 in fp32 for the mixedC2 plan).

Not a DevTools screenshot. CLOUD-HL, origin `http://localhost:8080`.

3.9 Resource monitor and decision engine

The decision engine (`engine/planner.js`) turns a device description into a plan in seven deterministic steps (Figure 23). **Step 1** reads the browser's signals: `deviceMemory`, `hardwareConcurrency`, the connection's downlink and round-trip time, the storage estimate, and the SHA-256 hashes already in the cache. The monitor deliberately does not infer the CPU architecture from the user-agent string (F-16). A **profile** can override any signal; profiles exist so that the engine can be tested deterministically for devices that were not available, and every value in the resulting context records whether it came from an override, the profile, the browser or a default. Table 12 lists the three profiles.

Table 12. Simulated device profiles (`engine/profiles.js`).

Profile	deviceMemory	Cores	Downlink / RTT	Allowance	Usable (−15%)	Accuracy floor	Weights m / t / l
low	1 GiB	2	1.5 Mbit/s / 300 ms	150 MiB	127.5 MiB	0.900	0.60 / 0.20 / 0.20
mid	4 GiB	4	10 Mbit/s / 100 ms	512 MiB	435.2 MiB	0.905	0.34 / 0.33 / 0.33
high	8 GiB	8	50 Mbit/s / 40 ms	1,536 MiB	1,305.6 MiB	0.910	0.10 / 0.60 / 0.30

Profile values replace the browser's signals; they are simulations, not measurements of real devices. Without a profile, the allowance is 12.5% of the browser-reported deviceMemory, capped at 2,048 MiB (`DEFAULTS` in the same file).

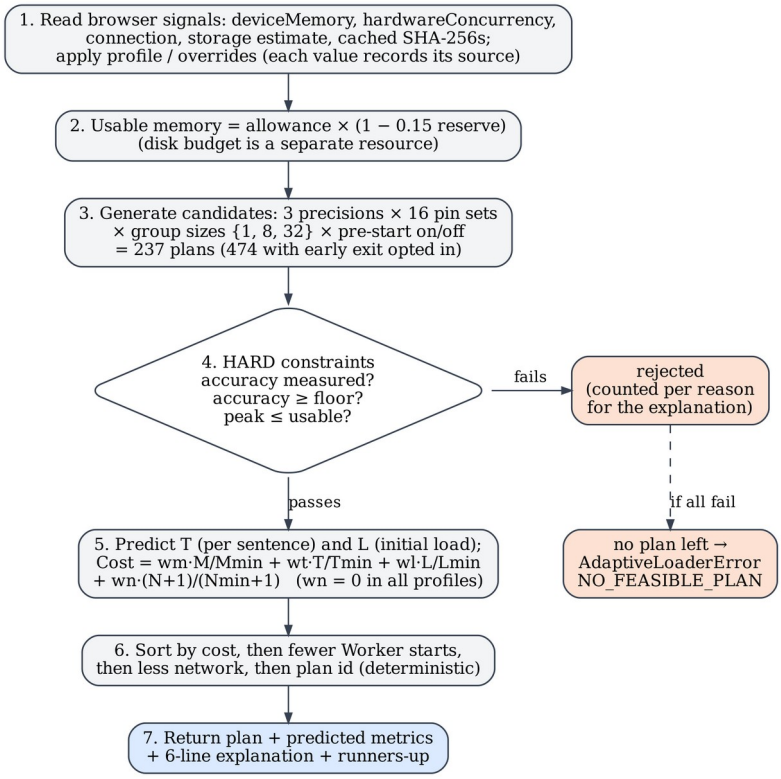


Figure 23. The seven steps of `decide()` (`engine/planner.js`).

Step 2 computes the usable memory budget: the allowance minus a 15% reserve. **Step 3** generates every candidate plan: 3 precision plans × 16 pinned sets, with group sizes {1, 8, 32} for non-resident plans and pre-start where at least two components are not pinned, giving **237 candidates**, or 474 when early exit is opted in, because each plan is then generated with and without early exit (Phase 5 report §5.2; re-counted from `generateCandidates()` on 1 November 2026: 79 plans per precision, made up of 6 sequential, 24 + 36 + 12 hybrid and 1 resident). **Step 4** applies the hard constraints before any cost is considered: a plan is rejected if its accuracy was never measured, if its measured validation accuracy is below the floor, or if its predicted peak memory exceeds the usable budget. Predicted memory uses the measured per-component heaps multiplied by an uncertainty factor of 1.10, plus the hidden states waiting in the page and a fixed overhead per Worker.

Step 5 predicts time and computes the cost (Figure 24): Cost = w_m·M/M_min + w_t·T/T_min + w_l·L/L_min + w_n·(N + 1)/(N_min + 1), where M is the predicted peak memory, T the predicted time per sentence, L the initial load, N the MiB to download on the first request, and each minimum is taken over the feasible plans; w_n is 0 in all three profiles. The time model adds the compute time of each component (weighted by the share of sentences that reach it when early exit is on), transfer time, and the reload time of non-pinned components divided by the group size; the initial load *L* is the load time on the first request's critical path, including download over the profile's link when a component is not

cached. The time inputs come from a per-device calibration or, by default, from cloud measurements. The calibration distinguishes **cold** session creation (no other component Worker alive) from **warm** creation, because creation was about 2.5 times slower when no other Worker was alive (F-28). The calibration needed three versions: the first was distorted by an outlier sample and by warm-up runs, and the second still measured creation only with another Worker alive, so it under-predicted sequential plans by 28–56% (Phase 5 report §6). **Step 6** sorts the feasible plans by cost and breaks ties deterministically, and **Step 7** returns the plan with its predicted metrics, the runners-up and a six-line explanation.

```
54      // 4 — HARD constraints, checked before any cost is considered
55      if (acc == null) { rejected.unverified++; if (rec) records.push({ ...rec,
status: 'rejected', reason: 'unverified', detail: 'accuracy never measured' });
continue; }
56      if (acc < ctx.minAccuracy) { rejected.accuracy++; if (rec)
records.push({ ...rec, status: 'rejected', reason: 'accuracy', detail: `${(acc *
100).toFixed(2)}% < ${(ctx.minAccuracy * 100).toFixed(1)}% floor` }); continue; }
57      if (mem.peakMiB > budget.usableMiB) { rejected.memory++; if (rec)
records.push({ ...rec, status: 'rejected', reason: 'memory', detail: `$
{mem.peakMiB.toFixed(1)} MiB > ${budget.usableMiB.toFixed(1)} MiB usable` }); continue;
}
    …
69    // 5 — normalised weighted cost (each term divided by the best feasible value)
70    const best = k => Math.min(...feasible.map(f => k(f)));
71    const mMin = best(f => f.mem.peakMiB), tMin = best(f => f.time.perSentenceMs),
lMin = best(f => f.time.initialLoadMs),
72          nMin = best(f => f.time.networkMiBFirstRequest);
73    const w = ctx.weights;
74    for (const f of feasible) {
75      f.terms = { m: f.mem.peakMiB / mMin, t: f.time.perSentenceMs / tMin, l:
f.time.initialLoadMs / lMin,
76                  n: (f.time.networkMiBFirstRequest + 1) / (nMin + 1) };
77      f.cost = w.m * f.terms.m + w.t * f.terms.t + w.l * f.terms.l + (w.n ?? 0) *
f.terms.n;
78    }
79    // 6 — deterministic order: cost, then preferences (fewer Worker startups, less
network), then id
80    feasible.sort((a, b) => (a.cost - b.cost) || (a.time.startupsPerSentence -
b.time.startupsPerSentence)
81                         || (a.time.networkMiBFirstRequest -
b.time.networkMiBFirstRequest) || a.plan.id.localeCompare(b.plan.id));
```

Figure 24. Hard constraints first, then the min-normalised weighted cost and the deterministic order.

Source: `packages/adaptive-onnx-loader/src/engine/planner.js`, lines 54–57, 69–81.

Table 13 shows the plans chosen with the default cloud calibration. The explanation for the mid profile, as produced by the engine, is visible in Figure 25: of 237 candidates, 76 were rejected for memory and 79 for accuracy, and mixedC2 resident had the lowest cost (1.310). The runner-up and its cost depend on the cache contents at the time of the decision; in this capture the cache already held the int8 components from the Phase 4 capture, and the next plan cost 2.816. In browser validation, all six decided plans and three forced plans produced outputs identical to the reference; the measured peak was within the usable budget for every

decided plan; memory was never under-predicted; and the time error of decided plans was −12.3% to +3.9% (Phase 5 report §3.2). Hybrid and pre-start plans were already 23–35% optimistic (F-32); Phase 7 measured this more precisely.

Table 13. Plans chosen by the engine (cloud calibration; Phase 5 report §3.1).

Profile	Sentences per request	Chosen plan	Rejected (memory / accuracy)	Feasible
low	1	int8 · sequential · G 1	234 / 0	3
low	32	int8 · sequential · G 32	234 / 0	3
mid	1	mixedC2 · resident	76 / 79	82
high	1	fp32 · resident	0 / 158	79
low + early exit (floor 0.895)	32	int8 · sequential · G 32 · early exit	468 / 0	6
mid + early exit	1	mixedC2 · resident · early exit (90.60% ≥ 0.905)	162 / 158	154

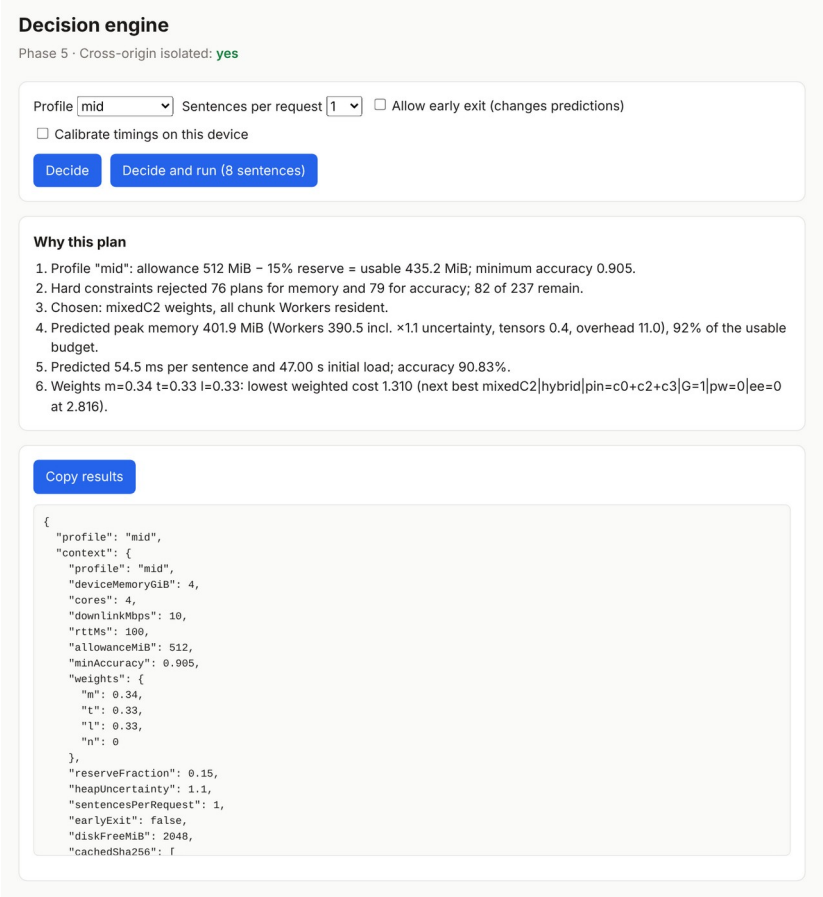


Figure 25. Decision-engine page (`demo/phase5-engine.html`), mid profile, default timings: the six-line explanation and the start of the decision record.

Documentation capture, CLOUD-HL.

3.10 Early exit and mixed precision

Both options trade accuracy for another resource, so the engine treats them through the same hard constraint: a plan may be chosen only if its **measured** SST-2 validation accuracy in the WebAssembly engine is at least the profile's floor. Table 14 is the accuracy table the engine uses (`engine/measurements.js`, Phase 5 report §3.3).

Table 14. Validation accuracy used by the hard constraint (WASM engine, 872 sentences, τ = 0.95).

Precision plan	Without early exit	With early exit	Exit after C1 / C2 / full
fp32	91.06%	90.71% (−0.35)	20.1% / 24.0% / 56.0%
mixedC2 (C2 fp32)	90.83%	90.60% (−0.23)	17.2% / 25.1% / 57.7%
int8	90.02%	89.91% (−0.11)	17.2% / 24.0% / 58.8%

The exit heads run inside the Workers of C1 and C2, in the same WebAssembly instance, so they add no Worker; in Phase 5 they added no measurable WebAssembly memory (355.06 MiB with and without exits; F-31). Early exit is a request, not a guarantee. Figure 26, from the Phase 6 demonstration, shows the high profile with early exit requested: every early-exit plan has at most 90.71% accuracy (fp32), below the 91.0% floor, so all of them are rejected and the engine runs full inference and says why. The mixedC2 setting follows from the sensitivity test of Section 3.4: keeping only C2 in fp32 reached 90.83%, recovering 78% of the gap between int8 and fp32 for 141 MiB more resident memory (355 vs 214 MiB), and it became the mid profile's choice (F-30).

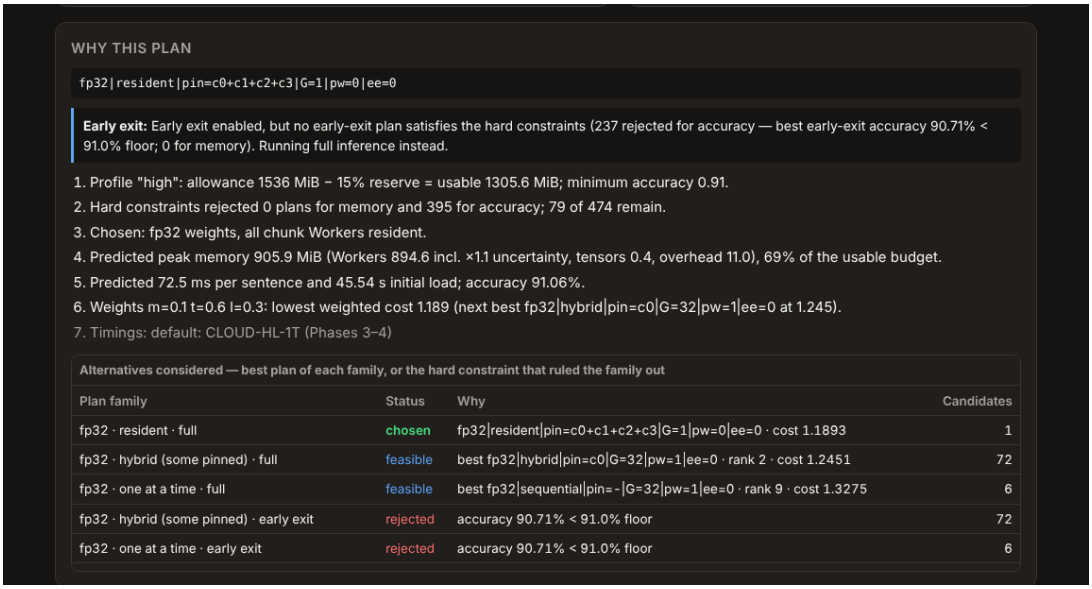


Figure 26. High profile with early exit requested: early-exit plans are rejected by the accuracy floor and the explanation says so.

Cropped from `docs/images/demo_high_earlyexit_requested.png` (Phase 6 demonstration screenshot).

3.11 Package and public API

Phase 6 packaged the library as `adaptive-onnx-loader` version 0.1.0 (MIT licence, author Moulieswaran M) with TypeScript declarations (`index.d.ts`) and `onnxruntime-web` 1.20.1 as an exact peer dependency. The package was built and tested as a tarball (`npm pack`: 30 files, 33.8 KiB at the first packaging test; 36.1 KiB after the later README and instrumentation changes) and installed by name into an empty project; it has **not been published** to the npm registry, which was left as the author's decision (Phase 6 report §4.1 and issue I-6.4). The public API is small (Table 15) and every failure is an `AdaptiveLoaderError` with one of seven codes (Table 16). A minimal use is shown in Figure 27.

Table 15. Public API (`index.d.ts`, `src/api.js`).

Member	Purpose
`loadModel(options)`	Validate options and environment, load the manifest and vocabulary (and `early_exit.json` if early exit is requested), open the store, decide a plan; returns an `AdaptiveModel`
`infer(text)` / `infer(texts[])`	Classify one sentence or an array; returns `{ label, score, logits, exitAt }`; calls are serialised through a promise queue
`explainPlan({ full? })`	The current plan, its explanation, predicted metrics, budget, candidates and runners-up
`previewPlan(changes)`	What the engine would choose with other options, without changing the running plan
`getCacheStats()`	IndexedDB entries, usage and quota; live Workers, live and peak WASM memory, Worker starts
`replan(changes)`	Decide again with a new profile, overrides or early-exit option
`clearCache()` / `dispose()`	Empty the store / terminate every Worker

Table 16. `AdaptiveLoaderError` codes (Phase 6 report §2).

Code	Raised when
`BAD_OPTIONS`	Missing or invalid `manifestUrl`, `infer()` given something other than strings, or no `ortUrl` outside a page
`NOT_ISOLATED`	The page lacks COOP/COEP; the message names the two headers
`NO_WORKERS`	Web Workers are not available
`MANIFEST`	The manifest cannot be fetched or parsed
`NO_FEASIBLE_PLAN`	No plan satisfies the constraints; the message is the engine's explanation
`EARLY_EXIT_UNAVAILABLE`	Early exit requested without its metadata
`DISPOSED`	The model has been disposed

```
35    import { loadModel } from 'adaptive-onnx-loader';
36
37    const model = await loadModel({ manifestUrl: '/models/onnx/manifest.json',
profile: 'mid' });
38    console.log(model.explainPlan().explanation.join('\n'));   // why this plan
39
40    const r = await model.infer('a gorgeous, witty, seductive movie.');
41    console.log(r);   // { label: 'POSITIVE', score: 0.99…, logits: [...], exitAt:
'full' }
42
43    console.log(await model.getCacheStats());
44    model.dispose();  // terminates every chunk Worker -> memory returned
```

Figure 27. Minimal use of the public API, from the package README. Source: `packages/adaptive-onnx-loader/README.md`, lines 35–44.

The package also exports the lower-level classes (tokenizer, sessions, runners, store, engine functions) for experiments; the README marks them as “Advanced”. The model-specific parts are explicit: the per-component heaps, timings and accuracy table in `engine/measurements.js` were measured for DistilBERT-SST2, so the package as built only “knows” this model (issue I-6.1). The demonstration page (`demo/index.html`, Figure 28) uses only the public API. It shows the decision and its alternatives, predicted against measured memory, the cache contents, and a comparison of the plans the engine would choose for each profile.

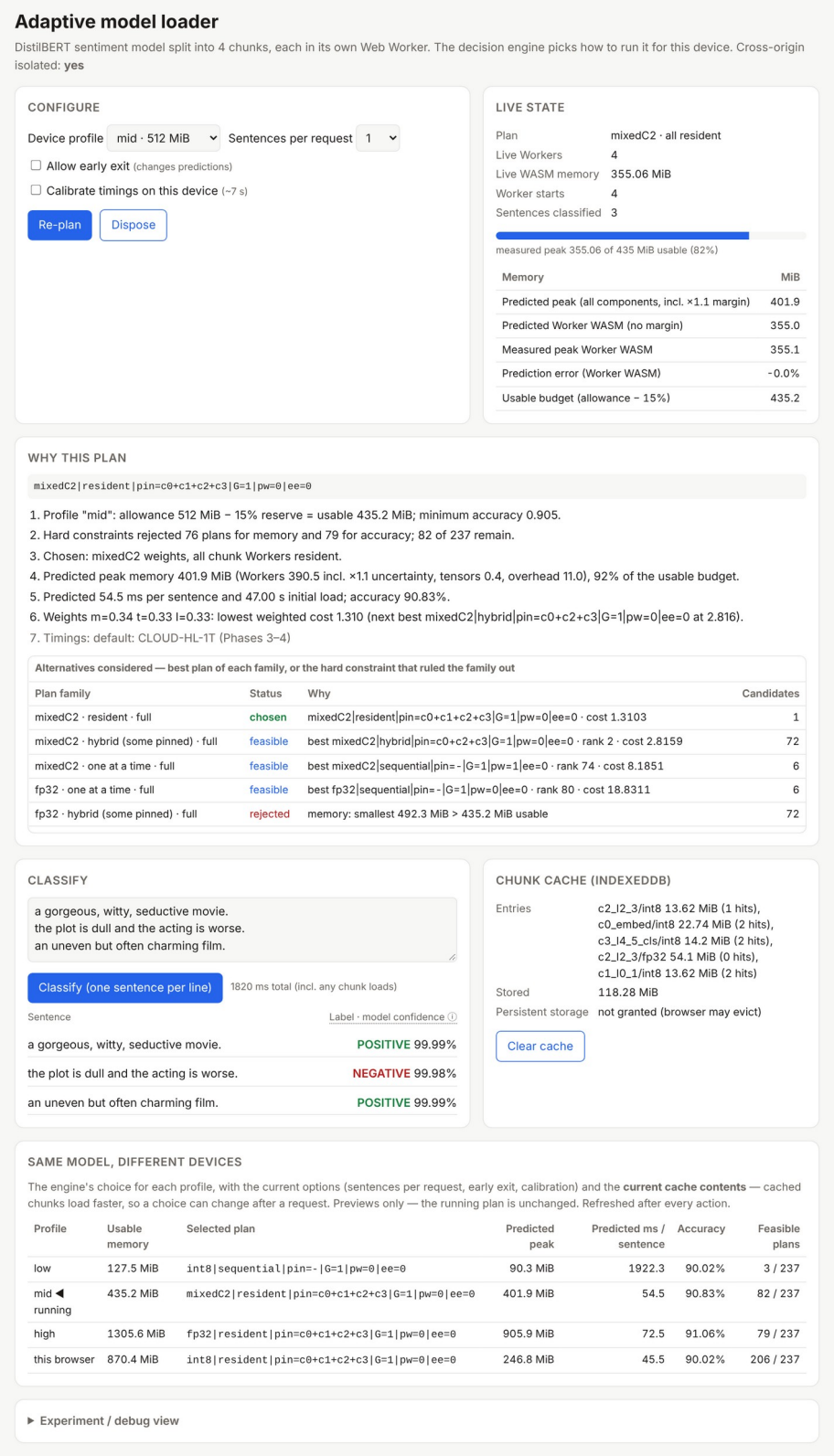


Figure 28. Demonstration page (`demo/index.html`) with the mid profile after classifying three sentences: live state, explanation, alternatives, cache and per-profile comparison.

Documentation capture, CLOUD-HL; the "this browser" row reflects the headless cloud browser's signals.

3.12 Testing and verification

Testing was layered. **Unit tests** (`node --test`) cover the tokenizer, the chaining rule and tensors, the cache (hit, miss, eviction, quota and corruption paths, using `fake-indexeddb`), the engine (profile-to-plan tables, constraints, determinism, and that tracing does not change the decision) and the API. **Browser gates** for each phase drive the real pages in headless Chromium and check outputs against reference logits, memory, cache state and errors. A **consumer test** installs the packed tarball into a fresh project and drives it through the public API only. **Manual checks** on the Mac repeated the key pages on a real device. Table 17 summarises the results.

Table 17. Verification by phase.

Phase	Automated checks	Result	Source
1	Split equivalence, parameter count, ONNX equivalence, per-chunk sensitivity, ORT-web smoke test	V1–V10 passed	Phase 1 report §7
2	Tokenizer unit tests (26 edge cases, 872 sentences); browser baseline test	4/4 unit; 20/20 browser checks	Phase 2 report §8
3	Browser gates (chain, Workers, memory proof); regression of Phase 2	48/48 gates, 8/8 unit, 5/5 regression	Phase 3 report §9
4	Cache unit tests; int8 and fp32 browser suites; throttled tests; Phase 2–3 regressions	17/17 unit; 26/26 + 26/26; 6/6; 23/23 + 5/5	Phase 4 report §9
5	Engine unit tests; browser validation of decided and forced plans	31/31 unit; 53/53 gates	Phase 5 report §9
6	Packaging; installed-tarball browser test; unit tests	8/8; 22/22; 40/40	Phase 6 report §4, §9
Report	Unit tests re-run on 1 Nov 2026 before writing (Node 22.23.4, Mac Linux sandbox)	40/40 pass	`npm test` output

Two API defects were found by the consumer test in Phase 6 and fixed with new unit tests (Phase 6 report §4.3). Every phase report also lists its open issues and deviations; these are summarised, with the Phase 7 deviations, in Section 3.13.6 and Appendix A.

3.13 Experimental evaluation

Phase 7 tested the system’s claims with experiments whose hypotheses, predictions, metrics and analysis rules were written down before the full suite was run. This section describes the design (3.13.1) and its execution (3.13.2), then reports the cloud results (3.13.3) and the Mac results (3.13.4) **separately**; the two machines differ in operating system, browser build, headless versus headed mode and protocol, so their numbers are never pooled or averaged. Section 3.13.5 sets the verdicts side by side and Section 3.13.6 collects the results that did not support the hypotheses or remain unexplained. Detailed tables are in Appendices B and C.

Results are classified with four labels, used consistently below: **supported** (the pre-registered criterion is met; for timings this means the 95% CI of the difference excludes 0, for memory a difference that was identical in every repetition); **not supported** (the criterion is not met); **descriptive only** (the pattern is present but no statistical test supports it, or the test is not significant); and **inconclusive** (the CI includes 0, or the data cannot decide).

3.13.1 Design

The experiment plan (`docs/PHASE_7_EXPERIMENT_PLAN.md`, version 1.2) fixed the following rules. Every result row names one **configuration** from `bench/configs.json` (Table 19). In the cloud, each run starts a fresh browser **process**; on the Mac each run reloads one tab. **Timing** configurations were repeated five times in a seeded random order. **Accuracy** was measured once per precision and early-exit setting on the 872 validation sentences, with a determinism repeat; every timing run also checked its outputs against the reference (int8 $\Delta = 0$, fp32 and mixedC2 $\Delta < 10^{-4}$, identical exit points). The **cache state** (off, cold or warm) was fixed per block and recorded in every row, and cold and warm results were never pooled. The workload was 100 sentences chosen at token-length quantiles (`results/phase7/workload_100.json`).

Six metrics were used: M1 bytes downloaded (server-side count), M2 time to first inference (TTFI), M3 peak memory (WebAssembly probe plus waiting hidden states; pre-start plans also report an upper bound from Worker lifetimes), M4 latency (median single-request latency for sentence-major plans after five warm-up requests, or amortised time per sentence for chunk-major plans, reported separately), M5 accuracy and M6 cache hit rate. **Statistics:** medians with bootstrap 95% confidence intervals from 10,000 resamples (Efron and Tibshirani 1993); no p-values were computed on n = 5 timings; accuracy pairs were compared with the exact McNemar test on the discordant predictions (McNemar 1947). A block was valid only if all its validation checks passed, and hypothesis outcomes were reported, never used to fail a block.

Table 18. Pre-registered hypotheses and predictions (experiment plan v1.2 §6).

ID	Supported if	Prediction recorded before the full suite
H1 memory policies	sequential < chain < baseline peak	Supported
H2 pre-start	pre-start lowers TTFI on slow links (CI excludes 0)	Not supported for TTFI; throughput benefit at G 32
H3 early exit	latency / compute lower; accuracy loss quantified; bytes lower (three separate verdicts)	Small latency benefit; bytes lower for early exits but higher for full-path sentences; accuracy −0.11 to −0.35 pt
H4 mixed precision	a mixed combination on the Pareto front between int8 and fp32	Supported; C2 the best single fp32 component
H5 cache	warm much faster than cold; speed-up grows as the link slows	Supported

ID	Supported if	Prediction recorded before the full suite
H6 int8 batch dependence	int8 flips grow with batch size / padding; fp32 invariant	Supported, platform-dependent
Engine	regret ≤ 10%, chosen plan in the measured top 3, 0 memory underestimates	Memory safe; pre-start time 23–37% optimistic

H1–H5 come from the Master Plan (§1); H6 and the engine criteria were added for Phase 7. The predictions were written after the pilot (one repetition per configuration) and before the full suite, so some of them, notably H2, were informed by the pilot.

Table 19. Experiment configurations (`bench/configs.json`; results report header).

Configuration	Machine and browser	Notes
CLOUD-HL-1T	Cloud x86-64, 2 vCPU; headless Chromium 153; 1 thread per session	Unthrottled localhost; one browser process per run
CLOUD-HL-1T-NET-{50, 20, 10, 5}	As above	Token-bucket server at 50 / 20 / 10 / 5 Mbit/s, first-byte delay 20 ms
NODE-WASM-X86	Cloud, Node, onnxruntime-web WASM	Offline accuracy passes (E4, E-B, F-5)
NATIVE-X86-ORT1.20.1 / ORT1.29.0	Cloud, Python ONNX Runtime (CPU)	Native reference; 1.29.0 supplementary
NATIVE-ARM64-ORT1.20.1	Mac's Linux sandbox (aarch64), Python ONNX Runtime	Native reference on ARM
MAC-CHROME-T1 / T2 / T4 / T8, W1, 4T	Apple Silicon Mac, Chrome 155, headed	One tab reloaded per run; localhost; no throttling

3.13.2 Pilot and execution

A **pilot** with one repetition per configuration was run first to test the harness. It found five defects, all fixed and re-run before the suite (Phase 7 pilot report §3.5). The most serious was P-1: in the bandwidth-limiting server written in Phase 4, the token bucket was smaller than one 64 KiB write below about 5.24 Mbit/s, so slow downloads stalled forever; the fix sets the capacity to at least 64 KiB, and the Phase 4 results, which used 20 Mbit/s, were unaffected. P-3 was a protocol error that would have made the engine's time predictions look 64–69% wrong because the harness loaded a sequential plan twice. The pilot also showed that Chrome's CPU throttling does not reach dedicated Workers (F-33), so simulated CPU throttling was dropped. Six further defects were fixed during the suite's smoke tests (S-1 to S-6), including 28 GB of temporary browser profiles that filled the disk and a random server port that made warm caches look empty, because IndexedDB is per origin (F-36).

The cloud suite then ran as thirteen validated blocks (Table 20): **1,310 browser runs and 249 offline passes, with 0 failed runs** and every output check passing. The Mac experiments ran on the student's Mac from a dedicated page. The first Mac run produced valid MAC-1 data, but from 03:51:37 UTC its Worker compute became 6–17 times slower

for the rest of the run, for a reason that was not determined; in addition, native aarch64 runs started by the assistant overlapped part of that run. That run's MAC-2 timings were therefore declared invalid, kept in full, and MAC-2, MAC-3 and MAC-4 were re-run on an idle Mac with a guard that paused the run whenever the page was hidden (Phase 7 Mac results §1, §7).

Table 20. Execution summary, Part A (cloud and offline; results report §1).

Step	Block	Runs	Validation gate	Result
1–2	native x86-64 (ORT 1.20.1 and 1.29.0); native aarch64 1.20.1	82 + 77 passes	files; batch-1 equals Phase 1	valid (max Δ 5.0e-7 on aarch64)
3	WASM (E4, E-B, F-5)	90 passes	batch-1 hash equals browser	valid
4–5	acc, ebrowser	12 + 2	determinism; browser = Node	valid
6	e1 (memory policies, early-exit latency)	100	cache off; outputs = reference; UA check	valid
7	f28 (session creation)	400	identical primed cache	valid
8	eng (decision engine)	200	warm 10 entries / 0 bytes; cold 0	valid
9	e3bytes	270	cold; exit path	valid
10	netcheck	4	throttle within ±5%	valid (+3.24 / +1.26 / +0.64 / +0.10%)
11–13	net, e5x, g1	200 + 120 + 2	cold / revisit / origin	valid

Totals: 1,310 browser rows, 249 offline passes, 0 failed runs, 0 retries; 904 rows carried an output check and all passed; wall-clock time 6.0 h.

3.13.3 Part A: cloud and offline results

E1, memory policies (H1). With the cache off and the same 100-sentence workload, peak memory was identical in all five repetitions of every configuration (Figure 29; Appendix B, Table 29). Loading one component at a time (sequential, G 32) lowered the peak against the unsplit baseline by **55.4%** for int8 (180.25 → 80.46 MiB) and **56.8%** for fp32 (656.38 → 283.40 MiB); the ordering sequential < chain < baseline held for every precision. Resident Workers used more memory than the baseline (+18.8% int8, +23.9% fp32) because of the per-Worker runtime overhead. The cost of sequential loading was time: TTFI rose by 887 ms [809, 1,003] (+72%) for int8, 827 ms (+60%) for mixedC2 and 901 ms (+50%) for fp32 against resident plans. **H1 is supported.** The browser's own memory measurement was always 4.2–10.2 MiB above the probe, which is the JavaScript heap the probe does not see.

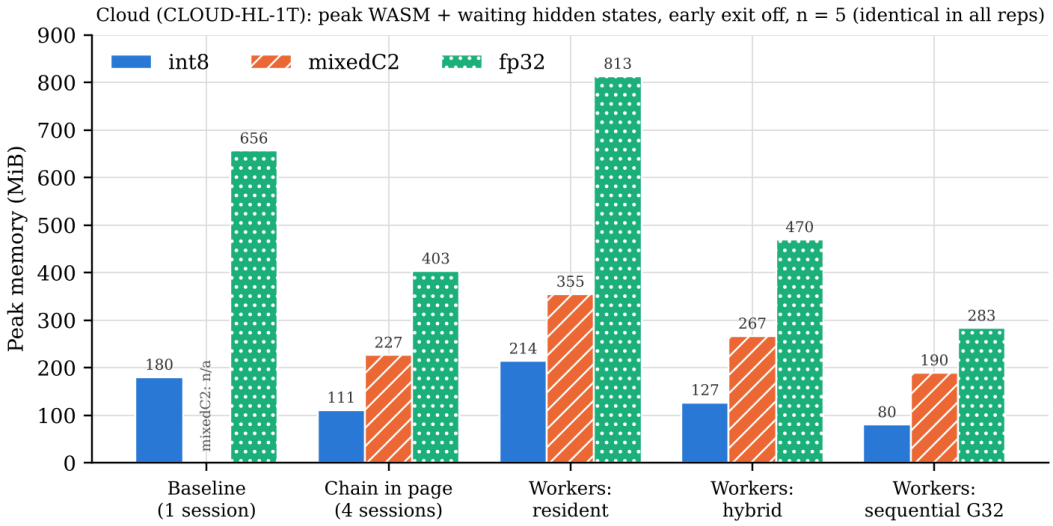


Figure 29. Cloud: peak memory by execution shape and precision (E1, `results/phase7/summary/analysis.json`, `E1.table[*].peakTotalMiB`).

E3, early exit (H3), three separate verdicts. *Accuracy:* with τ = 0.95 the loss was −0.11 (int8), −0.23 (mixedC2) and −0.34 points (fp32), and 41–44% of sentences exited early; none of the losses was statistically significant (McNemar p = 1.00, 0.63, 0.55), so the loss is quantified but not shown to be real at n = 872 (Table 21). *Latency:* in resident plans early exit lowered single-request latency by 18.6–20.4%, and in sequential plans the amortised time fell by 5.7–7.1%, every CI excluding 0 (Figure 30); reloads dominate sequential plans, so the gain there is smaller. Early exit added no memory. *Bytes:* for a single cold request, a sentence that exits at E1 needed 39.8–63.1% fewer bytes, but a sentence that runs the full path needed **1.8–7.0% more**, because the exit heads are downloaded too; pre-starting the next component cancelled most of the saving; and over a cached session the byte saving was about zero. **H3 is supported for latency, partly supported for bytes, and quantified but not significant for accuracy.**

Table 21. Early-exit accuracy on 872 validation sentences (results report §4.1).

Precision	Off	On (τ 0.95)	Δ points	McNemar flips, p	Exit at E1 / E2 / full
int8	90.02%	89.91%	−0.11	3 : 2, p = 1.00	150 / 209 / 513
mixedC2	90.83%	90.60%	−0.23	3 : 1, p = 0.63	150 / 219 / 503
fp32	91.06%	90.71%	−0.34	7 : 4, p = 0.55	175 / 209 / 488

Flips are counted as (correct with exit off, wrong with exit on) : (wrong off, correct on). Determinism: identical output hashes in two processes for all six settings.

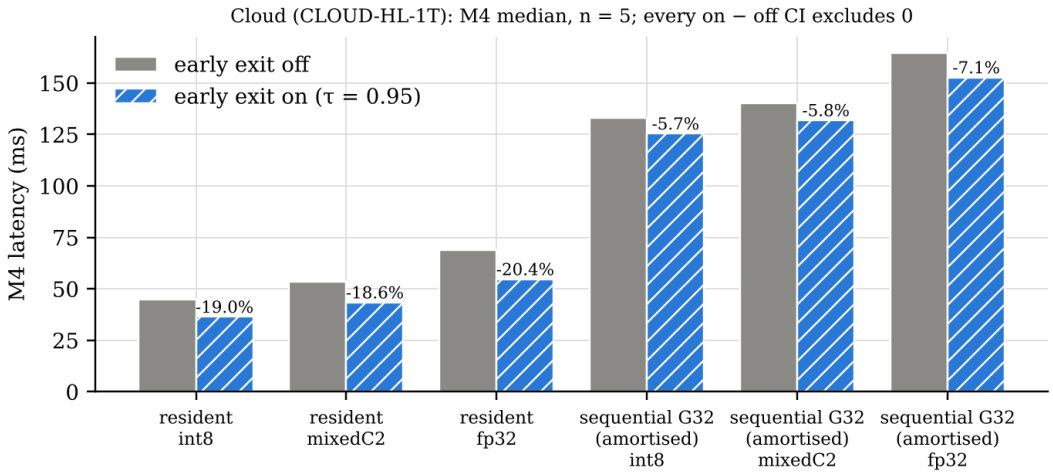


Figure 30. Cloud: latency with early exit off and on (E1 block, `analysis.json`, `E3latency`).

E2, pre-start (H2). Pre-starting the next component while the current one computes did **not** shorten TTFI. Across eight link and precision cells the largest effect was under 6% in either direction, the CI included 0 in six cells, and in one cell (mixedC2, unthrottled) pre-start was **slower** by 152 ms [57, 286] (+5.7%); on the 20 and 5 Mbit/s links the effect was at most 0.1%. Pre-start did reduce the time for the whole 100-sentence workload by 17.8–26.6% (CI excluding 0 in all eight cells), at a memory cost of +18–33% on the probe and up to +67% on the upper bound (fp32: 280 → 466 MiB). **H2 is not supported** for its stated outcome; the throughput benefit is a separate finding (F-40; Appendix B, Table 32).

E5, cache revisits (H5). Every revisit was served with **0 bytes** downloaded and a 100% IndexedDB hit rate. The speed-up of TTFI from cold to revisit grew as the link slowed, from ×1.16–1.34 on localhost to ×52 (int8) and ×78 (mixedC2) at 5 Mbit/s (Figure 31, Table 22). The 20 Mbit/s values reproduced Phase 4 (×13.5 and ×39.4 now; ×12.8 and ×37.8 then). On a revisit, 71–84% of TTFI was Worker start and session creation. **H5 is supported.**

Table 22. Cache: TTFI cold → revisit, median, speed-up [95% CI] (results report §6).

Link	int8	mixedC2	fp32
unthrottled	2,483 → 2,142 ms, ×1.16 [1.11, 1.20]	2,670 → 2,239, ×1.19 [1.13, 1.23]	3,809 → 2,840, ×1.34 [1.28, 1.38]
50 Mbit/s	12,807 → 2,110, ×6.07 [5.93, 6.12]	19,814 → 2,285, ×8.67 [8.52, 8.78]	46,024 → 2,878, ×15.99 [15.90, 16.41]
20 Mbit/s	29,009 → 2,146, ×13.52 [13.33, 13.71]	46,160 → 2,221, ×20.79 [20.60, 21.24]	110,336 → 2,799, ×39.42 [38.34, 40.08]
10 Mbit/s	55,989 → 2,153, ×26.01 [25.75, 26.57]	90,153 → 2,246, ×40.13 [39.46, 40.59]	217,491 → 2,854, ×76.21 [75.44, 78.57]
5 Mbit/s	110,045 → 2,117, ×51.98 [51.15, 53.40]	178,203 → 2,282, ×78.08 [77.12, 79.67]	omitted (decision 4)

Configuration CLOUD-HL-1T-NET-. The throttle measured +3.2% above target at 50 Mbit/s, within the ±5% gate.*

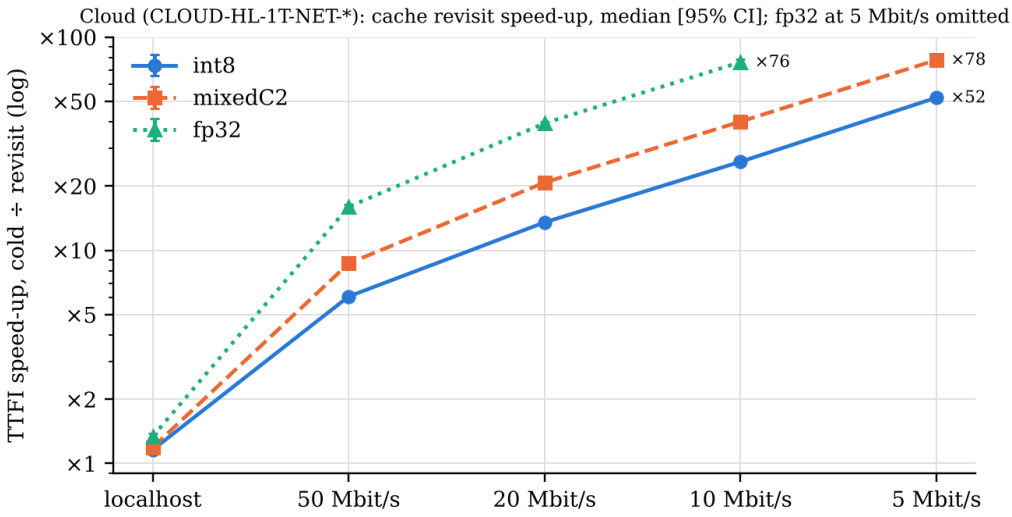


Figure 31. Cloud: cache revisit speed-up by link speed (`analysis.json`, `E5[*].speedup`, `speedupCI95`).

E4, mixed precision (H4). All 16 combinations of fp32 and int8 components were evaluated on the 872 sentences (Appendix B, Table 33). mixedC2 lies on the accuracy-versus-bytes Pareto front between int8 and fp32 (90.83% for 109.74 MB), and keeping C2 in fp32 recovered 77.8% of the int8-to-fp32 accuracy gap, more than any other single component. Unexpectedly, keeping only the embedding component C0 in fp32 **lowered** accuracy below all-int8 (−11.1% of the gap). The essential caveat is statistical: **no precision difference was significant at α = 0.05**, not even int8 against fp32 (14 : 5 discordant predictions, p = 0.064), and mixedC2 against int8 had p = 0.14. **H4 is supported descriptively only.**

E-B, int8 batch dependence (H6), and F-5, native versus WASM. int8 predictions changed with the batch on every platform (10–22 flipped sentences of 872 against batch 1, maximum logit change 2.4–3.5), while fp32 was invariant (0 flips). But the two sub-claims were **not supported**: the number of flips did not grow with batch size (batch 4 already flipped about as many as batch 32), and sorting sentences by length did not reduce it (equal or worse in 10 of 12 cells). Batch composition alone moved int8 accuracy by up to 1.6 points at a fixed batch size. **H6 is partly supported.** For native against WebAssembly int8 at batch 1, the difference depended on the platform: 1 flip on aarch64 with ORT 1.20.1 (exactly reproducing F-5), 0 flips on x86-64 with 1.20.1, and 4 flips with ORT 1.29.0; none was significant (results report §9; F-34). WebAssembly gave identical results everywhere it ran (Appendix B, Table 35).

E-F28, session creation. Over 400 creations, each in a fresh browser process, creating a session took 393 ms (int8) and 440 ms (fp32) when no other Worker existed, but **118 ms and 193 ms** (−70% and −56%) when another component Worker held a session, and about the same (119 / 203 ms) when that Worker held only a live WebAssembly instance with its

session released. Importing ORT's JavaScript alone did not help, and the effect ended when the other Worker was terminated. Creation is therefore fast only while another Worker holds a live, instantiated ORT WebAssembly module, which supports the hypothesis behind F-28 and explains why pinned and pre-started reloads are "warm" (results report §10).

E-ENG, the decision engine. One calibration was taken on the cloud machine, the engine made 12 decisions (3 profiles × early exit off/on × warm/cold cache), and 20 plans were each executed 5 times in both cache states. *Memory:* the prediction **never** underestimated the measured peak (0 of 200 runs); without the 10% margin it was exact (±0.1 MiB) for 12 of 20 plans, and the other 8, all pre-start or early-exit plans, were deliberately over-predicted. *Time:* the error depended on the plan shape (Figure 32, Table 23). Plain resident and sequential plans were within single digits when warm, but every plan with pre-start or a pinned subset was **28–52% optimistic**, and cold resident plans were 24% pessimistic. *Regret:* 10 of 12 decisions chose the best measured plan. The exception was the **mid profile without early exit**: the engine chose mixedC2 sequential G 32 with pre-start, which ranked 4th of 6 measured plans, with a regret of **24.0% warm and 42.9% cold**; the measured best was mixedC2 resident. The cause is the optimistic time prediction for the chosen shape (predicted 1,648 ms, measured 2,297 ms warm and 3,019 ms cold). **The engine criterion is not met for the mid profile.**

Table 23. Cloud: engine time-prediction error by plan shape, median [95% CI]; negative = optimistic (results report §11.2).

Shape	Warm cache	Cold cache
resident	−2.7% [−3.1, 0.9]	+23.7% [17.8, 30.6]
sequential	−6.2% [−7.4, −2.4]	−19.5% [−22.8, −12.7]
sequential + pre-start	−28.7% [−33.9, −27.5]	−45.4% [−48.5, −42.2]
hybrid	−30.3% [−35.5, −24.4]	−40.4% [−44.3, −35.3]
hybrid + pre-start	−39.7% [−41.1, −37.4]	−51.8% [−53.4, −49.8]

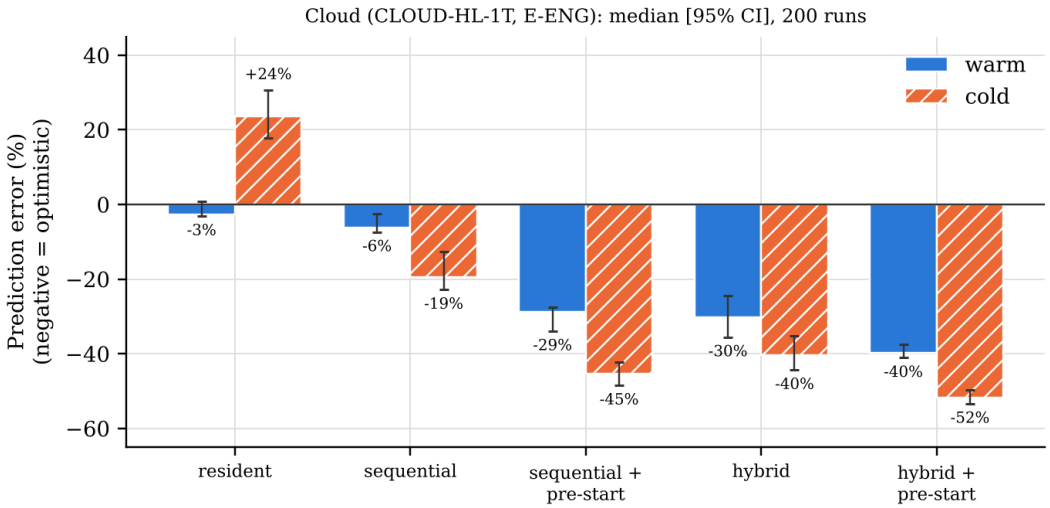


Figure 32. Cloud: engine time-prediction error by plan shape and cache state (`analysis.json`, `ENG.timeErrorByShapeAndCache`).

A refit of the time formula with three extra parameters was tried as a **proposal only**. Any fit that corrected the pre-start and hybrid plans made the resident plans, currently the best-predicted shape, worse (for example resident cold error from +20.4% to +33.0% or +56.0%), so the error comes from a cost the model does not contain rather than from badly set parameters. Neither fit was applied, the engine was not changed, and on 1 November 2026 the time-prediction model was recorded as **future work** (results report §11.4).

3.13.4 Part B: Mac results

The Mac results come from a different machine (Apple Silicon, Chrome 155, browser-reported 8 logical processors and `deviceMemory` 16), a headed browser, and a different protocol: one tab reloaded per run instead of a fresh process, localhost without throttling. Because a reload keeps the compiled WebAssembly module, Mac creation and cold-start times are **not comparable** with the cloud's (F-37). MAC-1 comes from the original run; MAC-2, MAC-3 and MAC-4 from the re-run; MAC-5 from a diagnostic page. All validation checks of the combined data passed, including 0 pauses of the foreground guard and stable CPU throughout the re-run (Table 24).

Table 24. Mac validation of the combined data (Mac results §1).

Check	Result
Original run complete (283/283 steps); re-run complete (235/235), MAC-2/3/4 only, not quick mode	passed
Foreground guard: every re-run row ran with the page visible, 0 visibility changes	passed (0 pauses, 0 anomalous rows)
Outputs equal the reference in every run that carries an output check: 268 runs = MAC-1 48 + MAC-3 40 + MAC-2 180 (the 10 MAC-4 session-creation rows run no inference and carry no output check); one origin; no failed steps	passed

Check	Result
Cache states: off = store unused; MAC-4 primed (10 entries); MAC-2 warm = 10 entries from IndexedDB, cold = all from network	passed
Expected repetitions: MAC-1 48, MAC-3 40, MAC-4 10, MAC-2 180	passed
CPU stability: 0 of 180 MAC-2 runs above 2× the first 20; maximum MAC-2 100.0 ms, MAC-3 56.1 ms (≤ 150 ms)	passed
Disclosed validator change: cache-off check changed from “store empty” to “store unused” after the first analysis	disclosed

MAC-1, thread sweep (findings F-13, F-43). On the Mac, int8 was **never faster** than fp32 (Figure 33): with 1, 2 and 8 page threads int8 was 9.6%, 8.7% and 19.4% slower, with CIs excluding 0, and with 4 threads the difference was −3.3% with a CI including 0 (inconclusive). Eight threads were slower than four (int8 +79%, fp32 +45%). The logits were identical at every thread count and equal to the cloud’s output hashes. On this device, int8 saves bytes and memory but not time; in the cloud int8 was faster (Phase 2: 43.1 against 70.2 ms with one thread).

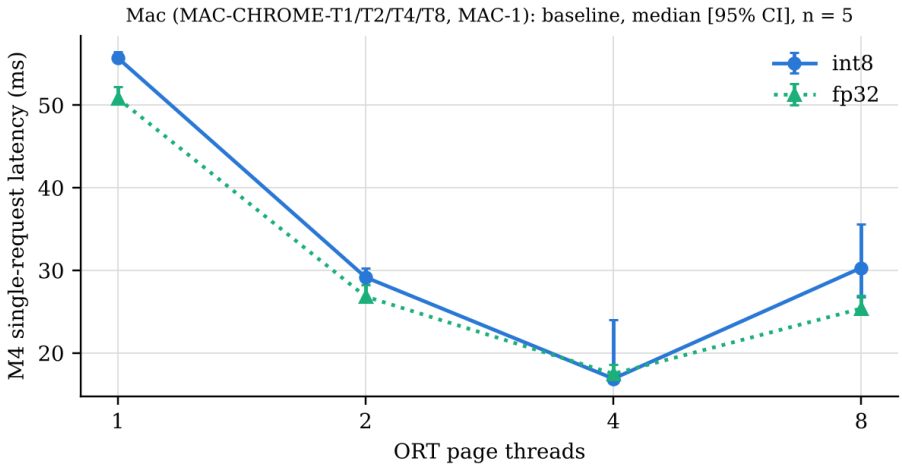


Figure 33. Mac: single-request latency against ORT page threads, int8 and fp32 (MAC-1; `analysis_mac_final.json`, `MAC1.table`).

MAC-3, policies and early exit. The peak memory of all eight plan and early-exit configurations was **identical to the cloud’s** to the last reported digit (for example 214.06 MiB resident int8, 80.46 MiB sequential int8), confirming F-27: memory results reproduce across platforms and only times vary. Early exit lowered latency on the Mac as well: −21.2% (int8) and −21.0% (mixedC2) in resident plans and −12.2% and −11.6% in sequential plans, every CI excluding 0 (Figure 34; Appendix C).

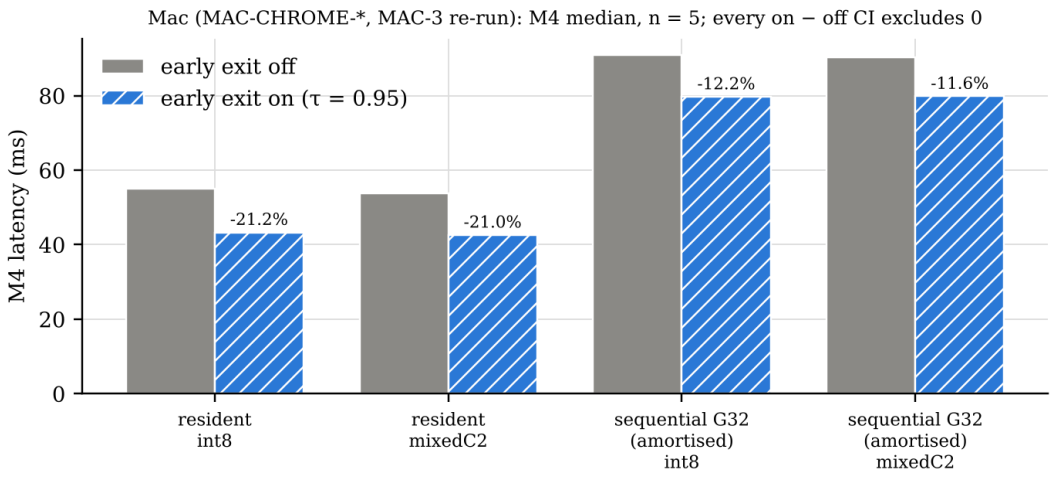


Figure 34. Mac: latency with early exit off and on (MAC-3 re-run; `analysis_mac_final.json`, `MAC3.table`).

MAC-4, session creation. With no other Worker alive, creation took 177.3 ms [174.3, 186.2]; with another component Worker alive, 62.5 ms [60.9, 65.5], a difference of −114.7 ms (−64.7%). The direction matches the cloud's E-F28; the absolute values are not comparable because of the reload protocol.

MAC-2, the engine on the Mac. The Mac calibration measured Worker start 13.3 ms, creation base 50.2 ms warm and 174.7 ms cold, and C1 compute 20.7 ms for int8 against 18.0 ms for fp32. With this calibration, 10 of 12 decisions equalled the cloud's; the mid profile with early exit chose mixedC2 sequential with pre-start and early exit instead of mixedC2 resident with early exit, a near-tie already noted in the cloud (F-35). *Memory:* 0 of 180 runs exceeded the prediction. *Time* (Figure 35, Table 25): the same pattern as the cloud, on a device whose browser reports 8 logical processors. Plain sequential and resident plans were accurate when warm, every pre-start or pinned shape was 15–29% optimistic, and cold resident plans were 124% **pessimistic**. The optimism was smaller than in the cloud, so part of it depends on the machine and it is not only contention on 2 vCPUs.

Table 25. Mac: engine time-prediction error by plan shape, median [95% CI] (Mac results §5.3).

Shape	Warm cache	Cold cache
sequential	+0.6% [−9.4, 5.6]	−1.9% [−13.3, −1.3]
resident	+5.3% [3.6, 7.1]	+124.0% [119.7, 124.3]
sequential + pre-start	−15.0% [−32.3, −10.2]	−22.7% [−40.8, −21.2]
hybrid	−19.6% [−24.0, −15.3]	−29.2% [−31.7, −27.0]
hybrid + pre-start	−17.8% [−20.8, −16.4]	−28.4% [−29.0, −26.4]

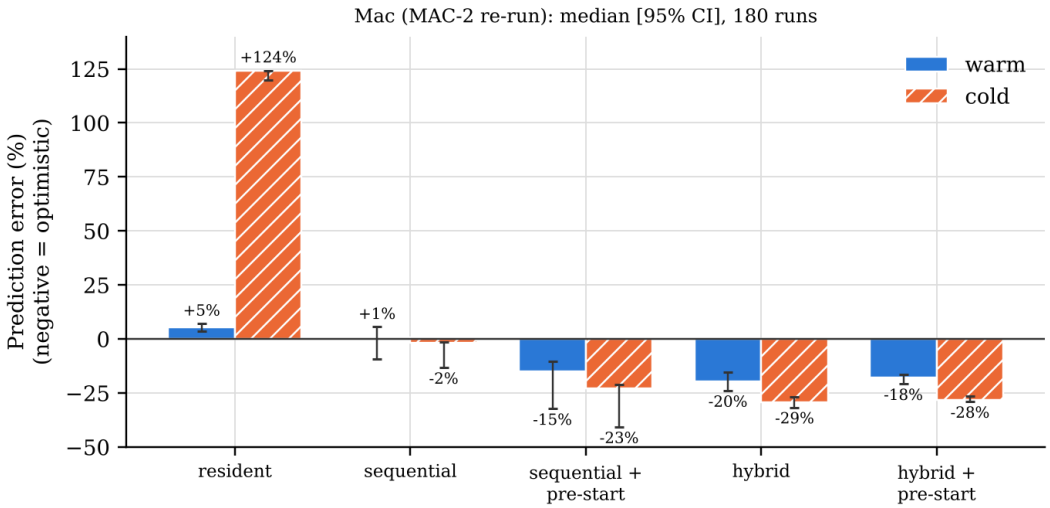


Figure 35. Mac: engine time-prediction error by plan shape and cache state (MAC-2 re-run; `analysis_mac_final.json`, `MAC2.timeErrorByShape`).

Regret on the Mac: in all 12 decisions the chosen plan was the best measured plan, with 0% regret (Spearman ρ between predicted and measured cost 0.2–1.0). This must be read with an important limitation: on the Mac each decision was measured against **its own top four plans by predicted cost**, whereas the cloud also measured the best plan of every family. mixedC2 resident, the cloud's measured best for the mid profile, was not measured on the Mac. The Mac result therefore means "best among the four plans the engine itself ranked highest"; it neither shows that the Mac avoids the cloud's mid-profile regret nor contradicts it.

MAC-5, the F-14 diagnostic. In Phase 2 the Mac had fetched the int8 model more slowly than the fp32 model (736.5 against 548.4 ms; F-14). In six rounds of a dedicated diagnostic this did **not reproduce**: int8 was faster in every round (70–241 ms against 196–356 ms). What did reproduce was a lower byte rate for int8 (1.1–3.7 times lower). F-14 remains an unexplained single observation.

3.13.5 Verdicts

Table 26 sets the verdicts side by side, with the cloud and the Mac in separate columns. A dash means that the experiment was not run on that machine; "device-independent" means that the quantity does not depend on the browser machine (accuracy is fixed by the WebAssembly kernels, which were bit-identical across platforms).

Table 26. Verdicts, cloud and Mac in separate columns (results report §15).

Hypothesis	Part A: cloud / offline	Part B: Mac	Verdict
H1 memory policies	sequential < chain < baseline (int8 80 < 111 < 180; fp32 283 < 403 < 656 MiB)	identical peaks	Supported

Hypothesis	Part A: cloud / offline	Part B: Mac	Verdict
H2 pre-start lowers TTFI	≤ 6% either way; CI includes 0 in 6 of 8 cells; one cell +5.7% slower; workload −18 to −27%	— (not run)	Not supported (throughput benefit found)
H3 latency / compute	−18.6 to −20.4% resident; −5.7 to −7.1% sequential	−21% resident; −12% sequential	Supported
H3 bytes	−40 to −63% (E1 exit), −9 to −20% (E2 exit), +1.8 to +7% full path; ≈ 0 per cached session	— (not measured)	Partly supported
H3 accuracy	−0.11 / −0.23 / −0.34 pt; p ≥ 0.55	device-independent	Quantified; not significant
H4 mixed precision	Pareto front; C2 recovers 77.8%; C0 fp32 lowers accuracy; mixed vs int8 p = 0.14	device-independent	Descriptive only
H5 cache	×1.16 to ×78; 0 bytes, 100% hits on revisit	— (not measured)	Supported
H6 int8 batch dependence	real on every platform; does not grow with batch; sorting does not help	— (native aarch64 is in Part A)	Partly supported (two sub-claims not supported)
F-5 native vs WASM	aarch64 1 flip; x86 ORT 1.20.1 0; x86 ORT 1.29.0 4; none significant	—	Reported per platform
Engine: memory	0 / 200 underestimates	0 / 180	Safe on both
Engine: time	pre-start / pinned −28 to −52%; cold resident +24%	pre-start / pinned −15 to −29%; cold resident +124%	Same pattern; future work
Engine: regret	10 / 12 rank 1; mid (no early exit) 24% / 43%	12 / 12 rank 1 within the top-4 predicted plans	Not met in the cloud; not tested with the same plan set on the Mac
E-F28 creation	(b) and (e) −56 to −70% vs (a)	(b) −64.7%	Supported
F-13 / F-43 int8 speed	int8 faster (cloud)	int8 never faster; 8 threads slower than 4	Device-dependent

3.13.6 Unsupported hypotheses, deviations and unexplained observations

The following results are reported as they were measured. None of them was removed, re-run selectively or re-analysed to change its outcome.

Table 27. Results that did not support the hypotheses, deviations and unexplained observations.

Item	What was observed	Status
H2 (pre-start)	No TTFI benefit; one cell 5.7% slower	Not supported (F-40)
H3 bytes	Full-path sentences need 1.8–7% more bytes; ≈ 0 saving over a cached session	Partly supported (F-41)
H3 accuracy	Early-exit accuracy loss −0.11 to −0.34 pt, p ≥ 0.55	Quantified, not significant (F-41)

Item	What was observed	Status
H4 and precision	No precision difference significant at n = 872 (int8 vs fp32 p = 0.064; mixedC2 vs int8 p = 0.14)	Descriptive only (F-42)
H6 sub-claims	Flips do not grow with batch size; length sorting does not help; composition moves accuracy by up to 1.6 pt	Not supported (F-38)
C0 in fp32	Lowers accuracy below all-int8	Unexpected; kept
Engine time model	Pre-start / pinned plans 28–52% (cloud) and 15–29% (Mac) optimistic; cold resident pessimistic; refit only moves the error	Future work (F-39); engine unchanged
Engine regret	Mid profile without early exit: rank 4 of 6, regret 24% warm / 43% cold (cloud)	Criterion not met
Mac regret scope	Measured on the top-4 predicted plans only	Limitation
Visible peak of pre-start plans	Same plan measured 652.6 MiB cold and 813.3 MiB warm; the upper bound is stable	Reported with bounds
F-14	int8 fetch slower than fp32 on the Mac in Phase 2; not reproduced in six rounds	Unexplained
I-4.8	Mac fp32 F4 run loaded from the network; narrowed to a 9–12 s window	Unexplained
Pilot fp32 baseline TTFI 11.1 s	Single pilot observation; not reproduced	Unexplained
First Mac run	Worker compute 6–17× slower from 03:51:37 UTC; overlapped by the assistant’s aarch64 runs	Cause not determined; data kept, not used for MAC-2/3/4
Validator changes	Mac cache-off criterion (“empty” → “unused”); refit target set included resident plans	Disclosed
Omissions	fp32 at 5 Mbit/s (decision 4); simulated CPU throttling (F-33); no throttled network on the Mac	Disclosed

Sources: results report §2, §12, §16; Mac results §1, §5.4, §6, §7; `docs/FINDINGS.md`.

4 Conclusion

4.1 Outcome

The project set out to test whether a transformer model in the browser can be managed as separately loaded components, so that memory use and download cost adapt to the device while accuracy stays within a stated bound. A complete system was built and evaluated for DistilBERT-SST2: an offline pipeline that splits the model losslessly into four ONNX components in fp32 and int8 with two early-exit heads; a browser runtime that runs each component in its own Web Worker and releases its memory by terminating the Worker; an IndexedDB cache keyed and verified by SHA-256; a deterministic decision engine that chooses a plan under memory and accuracy constraints and explains its choice; and a package with a small public API and a demonstration page that uses only that API. Across all experiments the outputs matched the reference in every checked run (904 of 904 cloud rows with an output check; 268 of 268 Mac runs with an output check, which excludes the 10 MAC-4 session-creation rows), and the 40 unit tests pass.

4.2 Major findings

The findings below are stated separately for the cloud machine (CLOUD-HL-1T) and the Mac (MAC-CHROME-*), with the strength of the evidence as classified in Section 3.13.

1. **Memory can be controlled by the execution shape** (supported). Loading one component at a time lowered peak memory by 55–57% against the unsplit model in the cloud, at a 50–72% higher time to first inference. The Mac produced identical peak values, so memory results reproduce across the two platforms while times do not.
2. **Releasing a session freed no memory; terminating its Worker did.** In the cloud browser's page-level memory measurement, `session.release()` freed 0% of the added memory and `worker.terminate()` returned the page to its 2.8 MiB starting level. This was measured in the cloud only and concerns the memory the browser attributes to the page, not memory returned to the operating system. It is the basis of the architecture.
3. **The cache removes the download on revisits** (supported). Revisits downloaded 0 bytes in the cloud; the TTFI speed-up grew from ×1.16–1.34 on localhost to ×52–78 at 5 Mbit/s. It was not measured on the Mac.
4. **Early exit lowers latency** (supported on both machines): −18.6 to −20.4% (cloud) and −21.0 to −21.2% (Mac) in resident plans, smaller in sequential plans. Its accuracy cost (−0.11 to −0.34 points) is **not statistically significant** at n = 872, and its byte saving holds only for single cold requests that exit early.

5. **Mixed precision is a descriptive improvement only.** mixedC2 is on the accuracy-versus-bytes Pareto front and recovers 77.8% of the int8–fp32 gap, but no precision difference is significant on 872 sentences.
6. **Pre-starting the next component does not shorten time to first inference** (H2 not supported, cloud), although it shortens a multi-sentence workload by 18–27% at a memory cost.
7. **The decision engine is memory-safe but its time model is incomplete.** Memory was never under-predicted (0 of 200 cloud runs, 0 of 180 Mac runs). Time was 28–52% (cloud) and 15–29% (Mac) optimistic for pre-start and pinned plans, which caused a 24–43% regret for the mid profile in the cloud. On the Mac every decision was best among the engine's own top four plans, a narrower test.
8. **Device and platform matter for int8 speed and native results.** int8 was faster than fp32 in the cloud but never faster on the Mac. Native ONNX Runtime int8 results differed by CPU architecture and runtime version, while the WebAssembly results were identical everywhere.

4.3 Limitations

The eleven limitations accepted at the end of Phase 7 (results report §16) apply to every result in this report:

1. **Two machines, not a controlled CPU experiment.** They differ in operating system, browser build, headless versus headed mode and protocol; simulated CPU throttling could not reach Workers (F-33).
2. **Mac protocol.** A tab reload keeps the compiled WebAssembly module (F-37), so Mac creation and cold-start times are not comparable with the cloud.
3. **Time model.** Optimistic for pre-start and pinned plans and pessimistic for cold resident plans on both machines; the refit only moved the error, so it was not applied.
4. **Mac regret** was measured on a narrower plan set (each decision only against the engine's own top four predicted plans); the mid-profile regret is confirmed in the cloud only.
5. **Statistical power.** 872 sentences cannot separate the precision settings (int8 vs fp32 $p = 0.064$) or the early-exit loss; five timing repetitions give wide intervals in some cells (for example Mac int8 with 4 threads [16.75, 24.06] ms).
6. **One model, task and data set.** The cost model and accuracy table are specific to DistilBERT-SST2, and τ was tuned on a development split of the training data.
7. **Network.** Links were simulated by a token-bucket server on localhost (+3.2% at 50 Mbit/s; first-byte delay 20 ms), fp32 at 5 Mbit/s was omitted, and the Mac had no throttled network.
8. **Memory probe.** It sees WebAssembly heaps only (JavaScript heaps added 4–10 MiB in spot checks); pre-start plans are reported with lower and upper bounds.

9. **Unexplained single observations** are kept: I-4.8, F-14, the pilot's 11.1 s fp32 baseline TTFI, and the first Mac run's slowdown.
10. **Native int8 results depend on the CPU and the ONNX Runtime version** (F-34); only the WebAssembly results are portable.
11. **No real low-memory device** was tested; profiles impose memory only as an engine budget.

In addition to the accepted list, the device signals available to a web page are coarse: `deviceMemory` and `hardwareConcurrency` are browser-reported values that are rounded or capped, so the engine's view of a real device without a profile is approximate (Section 2.8).

4.4 Future enhancements

- **Time-prediction model.** Identify and model the cost that is missing for pre-start and pinned plans (recorded as future work on 1 November 2026), then re-measure regret, including on the Mac with the same plan set as the cloud.
- **Real devices and networks.** Evaluate on real low-memory phones and over real networks rather than profiles and a simulated link.
- **More models and tasks.** Generalise the measurement step that produces the per-component heaps, timings and accuracy table, so that the package is not specific to one model; generative models would additionally need the key–value cache to be passed between components.
- **Statistical power.** Use larger evaluation sets to test whether the precision and early-exit accuracy differences are real.
- **WebGPU back-end,** once int8 operator support allows a fair comparison with WebAssembly.
- **Publishing the package** to the npm registry, which was left as the author's decision.

References

All online sources were accessed and verified on 1 November 2026.

- Devlin, J., Chang, M.-W., Lee, K. and Toutanova, K. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In: *Proceedings of NAACL-HLT 2019*. Association for Computational Linguistics. Available at: https://aclanthology.org/N19-1423/ [Accessed 1 November 2026].
- Efron, B. and Tibshirani, R. J. 1993. *An Introduction to the Bootstrap*. New York: Chapman & Hall.
- Haas, A., Rossberg, A., Schuff, D. L., Titzer, B. L., Holman, M., Gohman, D., Wagner, L., Zakai, A. and Bastien, J. F. 2017. Bringing the Web up to Speed with WebAssembly. In: *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2017)*. Available at: https://research.google/pubs/bringing-the-web-up-to-speed-with-webassembly/ [Accessed 1 November 2026].
- Hugging Face. 2026. distilbert/distilbert-base-uncased-finetuned-sst-2-english (model card). WWW document. Available at: https://huggingface.co/distilbert/distilbert-base-uncased-finetuned-sst-2-english [Accessed 1 November 2026].
- Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H. and Kalenichenko, D. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2018)*. Available at: https://openaccess.thecvf.com/content_cvpr_2018/html/Jacob_Quantization_and_Training_CVPR_2018_paper.html [Accessed 1 November 2026].
- Kang, Y., Hauswald, J., Gao, C., Rovinski, A., Mudge, T., Mars, J. and Tang, L. 2017. Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge. In: *Proceedings of the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2017)*. Available at: https://www.cl.cam.ac.uk/~ey204/teaching/ACS/R244_2017_2018/papers/kang_asplos_2017.pdf [Accessed 1 November 2026].
- McNemar, Q. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2), pp. 153–157. Available at: https://link.springer.com/doi/10.1007/BF02295996 [Accessed 1 November 2026].
- MDN Web Docs. 2026a. Web Workers API. WWW document. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API [Accessed 1 November 2026].

- MDN Web Docs. 2026b. Transferable objects. WWW document. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API/Transferable_objects [Accessed 1 November 2026].
- MDN Web Docs. 2026c. crossOriginIsolated global property. WWW document. Available at: https://developer.mozilla.org/en-US/docs/Web/API/CrossOriginIsolated [Accessed 1 November 2026].
- MDN Web Docs. 2026d. IndexedDB API. WWW document. Available at: https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API [Accessed 1 November 2026].
- MDN Web Docs. 2026e. SubtleCrypto: digest() method. WWW document. Available at: https://developer.mozilla.org/en-US/docs/Web/API/SubtleCrypto/digest [Accessed 1 November 2026].
- MDN Web Docs. 2026f. StorageManager. WWW document. Available at: https://developer.mozilla.org/en-US/docs/Web/API/StorageManager [Accessed 1 November 2026].
- MDN Web Docs. 2026g. Navigator: deviceMemory property. WWW document. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Navigator/deviceMemory [Accessed 1 November 2026].
- MDN Web Docs. 2026h. Navigator: hardwareConcurrency property. WWW document. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Navigator/hardwareConcurrency [Accessed 1 November 2026].
- MDN Web Docs. 2026i. NetworkInformation. WWW document. Available at: https://developer.mozilla.org/en-US/docs/Web/API/NetworkInformation [Accessed 1 November 2026].
- MDN Web Docs. 2026j. Performance: measureUserAgentSpecificMemory() method. WWW document. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Performance/measureUserAgentSpecificMemory [Accessed 1 November 2026].
- ONNX. 2026. Introduction to ONNX. WWW document. Available at: https://onnx.ai/onnx/intro/ [Accessed 1 November 2026].
- ONNX Runtime. 2026a. Get started with ONNX Runtime Web. WWW document. Available at: https://onnxruntime.ai/docs/get-started/with-javascript/web.html [Accessed 1 November 2026].
- ONNX Runtime. 2026b. Quantize ONNX models. WWW document. Available at: https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html [Accessed 1 November 2026].

- PyTorch. 2026. torch.onnx. WWW document. Available at: https://docs.pytorch.org/docs/stable/onnx.html [Accessed 1 November 2026].
- Sanh, V., Debut, L., Chaumond, J. and Wolf, T. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv:1910.01108. Available at: https://arxiv.org/abs/1910.01108 [Accessed 1 November 2026].
- Socher, R., Perelygin, A., Wu, J., Chuang, J., Manning, C. D., Ng, A. and Potts, C. 2013. Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank. In: *Proceedings of EMNLP 2013*. Available at: https://aclanthology.org/D13-1170/ [Accessed 1 November 2026].
- Sommerville, I. 2016. *Software Engineering*. 10th edn. Harlow: Pearson.
- Teerapittayanon, S., McDanel, B. and Kung, H. T. 2016. BranchyNet: Fast Inference via Early Exiting from Deep Neural Networks. In: *Proceedings of the 23rd International Conference on Pattern Recognition (ICPR 2016)*. Available at: https://arxiv.org/abs/1709.01686 [Accessed 1 November 2026].
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. and Polosukhin, I. 2017. Attention Is All You Need. In: *Advances in Neural Information Processing Systems 30 (NIPS 2017)*. Available at: https://proceedings.neurips.cc/paper/7181-attention-is-all [Accessed 1 November 2026].
- W3C. 2026. Device Memory API (Working Draft, 30 March 2026). WWW document. Available at: https://www.w3.org/TR/2026/WD-device-memory-1-20260330/ [Accessed 1 November 2026].
- Wang, A., Singh, A., Michael, J., Hill, F., Levy, O. and Bowman, S. 2018. GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding. In: *Proceedings of the 2018 EMNLP Workshop BlackboxNLP*. Available at: https://aclanthology.org/W18-5446/ [Accessed 1 November 2026].
- Xin, J., Tang, R., Lee, J., Yu, Y. and Lin, J. 2020. DeeBERT: Dynamic Early Exiting for Accelerating BERT Inference. In: *Proceedings of ACL 2020*. Available at: https://aclanthology.org/2020.acl-main.204 [Accessed 1 November 2026].

Appendix A Findings register

Condensed from `docs/FINDINGS.md` (one row per finding: the finding's headline sentence, the phase, and its status). Findings F-28 to F-32 were documented in the Phase 5 report (§4) but had not been transcribed into the register; on 1 November 2026 they were added with their finding text copied unchanged from that report (checked to be identical), phase 5, and the status "Observed", which was assigned at transcription because the Phase 5 table has no status column. The Phase 5 report itself is unchanged. The register keeps every entry; superseded or updated entries are marked there.

Table 28. Findings register (condensed).

ID	Finding (headline)	Phase	Status
F-1	Layer-group splitting is lossless: PyTorch chain Δ = 0.0, ONNX fp32 chain Δ = 2.4e-5, and the chunk files add up to the full model with no byte overhead	1	Observed
F-2	int8 sensitivity differs per chunk: C2 flips 20/872 predictions, C0 only 4/872	1	Observed → planned experiment
F-3	A model's WASM heap is ~2.6–2.8× its file size (fp32 255.5 MiB → 656 MiB; int8 64.2 MiB → 180 MiB)	2	Observed
F-4	The WASM heap does not shrink after `session.release()`.	2	Observed
F-5	Native vs WASM int8 kernels differ.	2	Mandatory in final report
F-6	With dynamic int8 quantization, a prediction depends on the batch.	1–2	Planned experiment E-B
F-7	`measureUserAgentSpecificMemory()` is missing from Playwright's headless shell, waits ~16 s for GC without `ForceEagerMeasureMemory`, and takes ~60 s per call once ORT worker threads exist	2	Observed
F-8	The WASM-heap probe agrees with the browser's whole-page measurement to within ~5 MiB	2	Observed
F-9	Load time on the shared cloud machine varies widely between runs (int8: 752 ms vs 6,365 ms for the same file)	2	Observed
F-10	Early exit after 4 layers keeps 91.06% accuracy at τ = 0.98 with 29% of sentences exiting early (τ swept on validation, so optimistic)	1	Observed → τ re-tuned on dev in Phase 5
F-11	(Phase 4: with the cache, the second load reads IndexedDB and downloads 0 bytes.) Reloading the same model in the same page after `release()` makes the fetch slower: 6/6 pairs (int8 ×1.2–1.4, fp32 ×2.2–2.7), while session creat…	2	Observed
F-12	The first large fetch in a fresh page pays a ~600 ms penalty that persists with a warm OS cache (int8: 987 / 793 ms vs ~150 ms afterwards)	2	Observed
F-13	On MAC-CHROME-4T, int8 latency ≈ fp32 (p50 16.0 vs 16.8 ms); in the cloud, int8 was clearly faster (1 thread 43 vs 70 ms; 2 threads 28 vs 37 ms)	2	Observed → Phase 7 thread sweep
F-14	On MAC-CHROME-4T, the int8 fetch (67 MB) took 736.5 ms vs 548.4 ms for fp32 (268 MB), a 5× lower byte rate.	2	Not reproduced

ID	Finding (headline)	Phase	Status
F-15	After loading, the JS heap still holds the downloaded model bytes (+67.4 / +268.0 MB, equal to the file size), still present when `release()` returns.	2	Observed
F-16	MAC-CHROME-4T reports `navigator.deviceMemory` = 16 (older specifications capped this at 8).	2	Observed
F-17	Splitting alone lowers peak WASM memory by 38% with all 4 chunks resident in one page (fp32 656 → 403 MiB; int8 180 → 111 MiB), at the same latency (p50 70.5 vs 70.2 ms; 42.0 vs 43.1 ms).	3	Observed
F-18	Each Worker has a fixed cost of ~16–19 MiB heap and ~300 ms WASM start-up (even for a 0.6 MB graph), and needs 2.9–3.4× its file size in heap.	3	Observed
F-19	Passing tensors between Workers with transferable buffers costs 0.1–0.4 ms per boundary, 1.17 ms (int8, 2.6%) / 1.34 ms (fp32, 1.8%) per sentence	3	Observed
F-20	The int8 chain is bit-identical to the full int8 model in WASM (Δ = 0, 872/872); cutting the graph doesn't change dynamic quantization	3	Observed
F-21	Sequential (chunk-major) mode holds every waiting sentence's hidden state in the main thread: 64.3 MiB for 872 sentences (~75 KiB each).	3	Observed → `groupSize` limit in Phase 5
F-22	Once bytes are cached, session creation dominates the load: warm request = 63% (fp32) to 72% (int8) session creation in the cloud, 66% / 79% on your Mac (MAC-CHROME-W1) (330–475 ms per chunk); bytes from IndexedDB including…	4	Observed
F-23	How much the cache helps depends on the link: localhost warm ×1.1 (int8) / ×1.6 (fp32); 20 Mbit/s: ×12.8 (int8, 29.1 → 2.27 s) / ×37.8 (fp32, 110.4 → 2.92 s); a revisit downloads 0 bytes.	4	Observed
F-24	LRU gets a 0% hit rate when the budget is smaller than the working set	4	Observed
F-25	SHA-256 verification runs at 0.4–1.1 GB/s: 13–39 ms (int8) / 63–115 ms (fp32) per chunk, ≈ 5–20% of a load, so verify-on-read stays on by default	4	Observed
F-26	Headless Chromium: ~10 GiB quota, `persist()` = false, so the cache can be evicted by the browser; the loader recovers by re-downloading	4	Observed
F-27	WASM heap sizes are identical across platforms:	4	Observed on 2 platforms
F-28	Session creation is ~2.5× slower when no other chunk Worker is alive	5	Observed
F-29	Inference time depends on sentence length and on JIT warm-up:	5	Observed
F-30	Mixed precision (C2 fp32, rest int8) reaches 90.83% in WASM	5	Observed
F-31	Early exit at τ = 0.95 (tuned on dev)	5	Observed
F-32	The cost model's time error depends on plan shape.	5	Observed
F-33	Chrome's CDP CPU throttling does not reach dedicated Workers.	7 (pilot)	Proven
F-34	The native int8 reference depends on CPU architecture and ORT version; WASM does not.	7 (pilot)	Observed
F-35	Some engine decisions are near-ties that calibration noise can flip.	7 (pilot)	Observed
F-36	IndexedDB is per origin (scheme + host + port).	7 (suite smoke test)	Proven

ID	Finding (headline)	Phase	Status
F-37	Session creation is fast (≈100–180 ms) when the compiled ORT WebAssembly module is still around:	7 (pilot, Mac-page self-test; E-F28 n = 400)	Measured
F-38	Dynamic-int8 batch dependence is real but does not grow with batch size, and length-sorting does not reduce it.	7	Measured
F-39	The engine's time model is 28–52% optimistic for every plan with pre-start or a pin set	7	Measured on both machines
F-40	Pre-start does not lower time to first inference	7	Measured
F-41	Early exit saves bytes only for single cold requests that exit early	7	Measured
F-42	No precision difference is statistically significant on the 872-sentence validation set	7	Measured
F-43	On the Mac, int8 is never faster than fp32, and 8 page threads are slower than 4.	7 (Mac)	Measured
D-3	`terminate()` frees 100% of a Worker's memory; `session.release()` frees 0% (browser's own memory API: int8 2.8 → 223.4 → 223.5 → 2.8 MiB; fp32 2.8 → 822.7 → 822.7 → 2.8 MiB)	3	Proven

Appendix B Detailed Phase 7 statistics (Part A: cloud and offline)

The tables in this appendix are transcribed programmatically from the final results report, which was itself generated from the raw data by `bench/phase7/analyze.mjs`. Medians with [bootstrap 95% CI], n = 5 per timing configuration unless stated.

Table 29. E1: peak memory, TTFI and M4 latency by policy and precision (CLOUD-HL-1T, cache off).

Policy	Precision	Peak total MiB	vs baseline	vs chain	TTFI ms	M4 ms	Worker starts
baseline	int8	180.25	—	+62.1%	760 [717, 868]	42.07 [41.13, 43.88]	—
chain in page	int8	111.19	−38.3%	—	874 [826, 896]	42.14 [41.79, 42.22]	—
resident	int8	214.06	+18.8%	+92.5%	1,232 [1,183, 1,285]	44.91 [44.12, 45.77]	4
hybrid (pin C0, G 32)	int8	126.84	−29.6%	+14.1%	1,230 [1,226, 1,275]	70.17 [69.14, 70.95] (amort.)	16
sequential G 32	int8	80.46	−55.4%	−27.6%	2,119 [2,044, 2,196]	133.03 [132.27, 135.14] (amort.)	20
chain in page	mixedC2	226.75	n/a	—	967 [942, 1,051]	47.99 [47.24, 50.03]	—
resident	mixedC2	355.06	n/a	+56.6%	1,378 [1,318, 1,412]	53.28 [50.48, 55.53]	4
hybrid	mixedC2	266.71	n/a	+17.6%	1,356 [1,223, 1,418]	79.19 [76.01, 81.98]	16
sequential G 32	mixedC2	189.53	n/a	−16.4%	2,206 [2,067, 2,274]	140.03 [137.70, 142.00]	20
baseline	fp32	656.38	—	+62.7%	1,347 [1,287, 1,525]	65.88 [64.46, 67.49]	—
chain in page	fp32	403.44	−38.5%	—	1,385 [1,163, 1,574]	67.84 [63.17, 69.17]	—
resident	fp32	813.31	+23.9%	+101.6 %	1,804 [1,585, 1,904]	68.71 [67.46, 70.99]	4
hybrid	fp32	469.65	−28.4%	+16.4%	1,860 [1,784, 1,938]	100.48 [99.92, 102.05]	16
sequential G 32	fp32	283.40	−56.8%	−29.8%	2,705 [2,587, 2,907]	164.41 [160.32, 169.89]	20

Transcribed programmatically from `docs/phase-reports/PHASE_7_RESULTS_REPORT.md`, section “3. E1”.

Table 30. E3: early-exit latency and compute (CLOUD-HL-1T).

Plan	Precision	M4 off → on	Δ [CI]	Compute per sentence off → on	Δ [CI]
resident	int8	44.91 → 36.39 ms	−8.51 [−9.91, −5.22] (−19.0%)	44.59 → 35.68	−8.91 [−9.92, −7.93] (−20.0%)
resident	mixedC2	53.28 → 43.38	−9.90 [−12.15, −6.77] (−18.6%)	51.65 → 41.05	−10.60 [−13.23, −7.68] (−20.5%)
resident	fp32	68.71 → 54.66	−14.04 [−17.01, −10.41] (−20.4%)	68.22 → 53.78	−14.44 [−17.06, −11.38] (−21.2%)
sequential G 32	int8	133.03 → 125.43 (amort.)	−7.60 [−14.43, −5.16] (−5.7%)	53.93 → 45.04	−8.89 [−10.74, −7.29] (−16.5%)
sequential G 32	mixedC2	140.03 → 131.94	−8.09 [−15.51, −5.35] (−5.8%)	58.12 → 47.86	−10.26 [−11.37, −8.80] (−17.7%)
sequential G 32	fp32	164.41 → 152.74	−11.67 [−17.33, −7.58] (−7.1%)	64.30 → 52.94	−11.36 [−15.19, −10.19] (−17.7%)

Transcribed programmatically from `docs/phase-reports/PHASE_7_RESULTS_REPORT.md`, section "4.2 Latency".

Table 31. E3: bytes for single cold requests by exit class (CLOUD-HL-1T).

Precision	Sentence exits at	Early exit off	On	On + pre-start
int8	E1	67.30 MB	40.50 (−39.8%)	57.15 (−15.1%)
int8	E2	67.30	57.15 (−15.1%)	72.04 (+7.0%)
int8	full	67.30	72.04 (+7.0%)	72.04 (+7.0%)
mixedC2	E1	109.74	40.50 (−63.1%)	99.60 (−9.2%)
mixedC2	E2	109.74	99.60 (−9.2%)	114.48 (+4.3%)
mixedC2	full	109.74	114.48 (+4.3%)	114.48 (+4.3%)
fp32	E1	267.90	154.45 (−42.4%)	213.54 (−20.3%)
fp32	E2	267.90	213.54 (−20.3%)	272.64 (+1.8%)
fp32	full	267.90	272.64 (+1.8%)	272.64 (+1.8%)

Transcribed programmatically from `docs/phase-reports/PHASE_7_RESULTS_REPORT.md`, section "4.3 Bytes".

Table 32. E2: pre-start, TTFI and 100-sentence workload (CLOUD-HL-1T-NET-*).

Link	Precision	TTFI off → on (ms)	Δ TTFI [CI]	Workload off → on (ms)	Δ workload [CI]	Peak probe off / on	Upper bound off / on
unthrottled	int8	2,483 → 2,417	−67 [−165, 54] (−2.7%)	13,395 → 9,832	−26.6% [−3,988, −3,140]	80.46 / 94.90	77.2 / 122.5
unthrottled	mixedC2	2,670 → 2,822	+152 [57, 286] (+5.7%)	14,436 → 11,329	−21.5% [−3,806, −2,432]	189.53 / 235.90	186.3 / 232.7
unthrottled	fp32	3,809 → 3,816	+7 [−149, 278] (+0.2%)	17,880 → 14,700	−17.8% [−4,588, −2,065]	283.40 / 375.78	280.1 / 466.4

Link	Precision	TTFI off → on (ms)	Δ TTFI [CI]	Workload off → on (ms)	Δ workload [CI]	Peak probe off / on	Upper bound off / on
20 Mbit/s	int8	29,009 → 28,982	−27 [−151, 97] (−0.1%)	13,412 → 10,186	−24.1% [−3,343, −2,818]	same	same
20 Mbit/s	mixedC2	46,160 → 46,111	−49 [−185, −13] (−0.1%)	14,206 → 10,985	−22.7% [−3,747, −2,805]	same	same
20 Mbit/s	fp32	110,336 → 110,299	−37 [−172, 58] (0.0%)	17,899 → 14,268	−20.3% [−4,381, −2,549]	same	same
5 Mbit/s	int8	110,045 → 109,925	−120 [−584, 22] (−0.1%)	13,405 → 9,963	−25.7% [−3,756, −2,966]	same	same
5 Mbit/s	mixedC2	178,203 → 178,295	+92 [−125, 195] (+0.1%)	14,238 → 11,311	−20.6% [−3,413, −2,511]	same	same
5 Mbit/s	fp32	*omitted (decision 4)*					

Transcribed programmatically from `docs/phase-reports/PHASE_7_RESULTS_REPORT.md`, section “5. E2”.

Table 33. E4: all 16 precision combinations (NODE-WASM-X86, 872 sentences).

Spec (C0 C1 C2 C3)	Accuracy	Flips vs fp32 / int8	McNemar vs int8 p	vs fp32 p	MB	Resident heap MiB
f f f f (fp32)	91.06%	0 / 19	0.064	—	267.90	813.3
i i f i (mixedC2)	90.83%	12 / 17	0.143	0.774	109.74	355.0
i i f f	90.83%	12 / 19	0.167	0.774	153.96	469.3
f f f i	90.83%	2 / 21	0.189	0.500	223.69	699.0
i f f f	90.60%	4 / 17	0.332	0.125	196.40	610.4
f i f i, f i f f, f f i f	90.48%	11–23 / 16–28	0.45–0.57	0.23–0.41	181–225	558–672
i f i i, i f i f, i f f i, f f i i	90.37%	6–24 / 19–29	0.65–0.71	0.03–0.31	110–181	355–558
i i i f	90.25%	21 / 2	0.500	0.189	111.51	328.4
i i i i (int8)	90.02%	19 / 0	—	0.064	67.30	214.1
f i i f	90.02%	23 / 18	1.000	0.093	183.01	531.3
f i i i	89.91%	22 / 15	1.000	0.052	138.80	417.0

Transcribed programmatically from `docs/phase-reports/PHASE_7_RESULTS_REPORT.md`, section “7. E4”.

Table 34. E-B: int8 batch dependence, flips vs batch 1 (of 872) and accuracy.

Platform	Batch 1	Batch 4: order / sorted / shuffled	Batch 8	Batch 16	Batch 32
WASM (= browser; hash-checked at b1 and b32)	90.02%	18 / 15 / 13 [12–20] (89.79–90.48%)	13 / 18 / 17 [12–20] (89.45–90.60%)	16 / 18 / 14 [13–19] (89.22–90.60%)	16 / 18 / 14 [13–19]; order 90.71%
native x86-64 ORT 1.20.1	90.02%	16 / 12 / 14 [12–21]	14 / 20 / 16 [12–20]	16 / 20 / 16 [15–18]	16 / 19 / 16 [11–17]; order 90.71%
native aarch64 ORT 1.20.1	90.14%	15 / 15 / 15 [14–20]	13 / 17 / 16 [15–21]	16 / 22 / 16 [15–19]	10 / 13 / 17 [10–20]; order 90.37%
native x86-64 ORT 1.29.0 (suppl.)	90.02%	—	—	—	order 16 flips; 89.79%
fp32 control (all platforms)	91.06%	0 flips, max |Δ| ≤ 1.1e-5 (exactly 0 on native x86)	0	0	0

Transcribed programmatically from `docs/phase-reports/PHASE_7_RESULTS_REPORT.md`, section "8. E-B".

Table 35. F-5: native vs WASM at batch 1.

Native platform	Setting	Native	WASM	Flips	Mean |Δ|	Max |Δ|	McNemar p
aarch64 ORT 1.20.1 (= original F-5)	full int8	90.14%	90.02%	1	1.82e-2	1.215	1.00
	chain int8	90.14%	90.02%	1	1.82e-2	1.215	1.00
	mixedC2	90.60%	90.83%	2	7.99e-3	0.981	0.50
	fp32	91.06%	91.06%	0	8.9e-7	1.2e-5	1.00
x86-64 ORT 1.20.1	full / chain int8	90.02%	90.02%	0	1.05e-2	0.619	1.00
	mixedC2	90.71%	90.83%	1	3.93e-3	0.422	1.00
	fp32	91.06%	91.06%	0	8.4e-7	8.7e-6	1.00
x86-64 ORT 1.29.0 (suppl.)	full / chain int8	90.02%	90.02%	4	4.93e-2	2.193	1.00
	mixedC2	90.48%	90.83%	3	1.62e-2	0.981	0.25

Transcribed programmatically from `docs/phase-reports/PHASE_7_RESULTS_REPORT.md`, section "9. F-5".

Table 36. E-F28: session creation time by state (400 creations, n = 40 per cell).

State	int8	fp32	vs (a) int8 / fp32
(a) no other Worker	393 [379, 406]	440 [434, 447]	—
(b) another chunk Worker with a session	118 [115, 121]	193 [185, 201]	−276 [−287, −260] (−70%) / −247 (−56%)
(c) ORT's JS imported only	394 [382, 402]	445 [437, 448]	+1 [−16, 18] / +4 [−4, 12]
(d) other Worker just terminated	377 [368, 393]	431 [424, 440]	−16 [−31, 5] / −10 [−19, 1]
(e) live WASM instance, session released	119 [114, 129]	203 [195, 208]	−274 [−287, −257] (−70%) / −238 (−54%)

Transcribed programmatically from `docs/phase-reports/PHASE_7_RESULTS_REPORT.md`, section "10. E-F28".

Table 37. E-ENG: decisions, rank and regret (CLOUD-HL-1T).

Profile	Early exit	Chosen (both cache states)	Margin to #2	Rank among measured	Regret warm / cold	Spearman ρ (pred vs measured cost) warm / cold
low	off / on	int8 sequential G 32	13.35%	1 / 3	0% / 0%	1.00 / 1.00
mid	off	mixedC2 sequential G 32 + pre-start	2.60% / 1.91%	4 / 6	24.0% / 42.9%	0.43 / 0.54
mid	on	mixedC2 resident + early exit	0.31% / 0.91%	1 / 8	0% / 0%	0.83 / 0.71
high	off / on	fp32 resident (early exit rejected: accuracy floor)	9.32% / 9.95%	1 / 6	0% / 0%	0.94 / 0.83

Transcribed programmatically from `docs/phase-reports/PHASE_7_RESULTS_REPORT.md`, section "11.3 Decisions".

Table 38. E-ENG: I-5.1 refit, proposal only (not applied).

Shape (eval reps)	Engine	Fit A: all affected plans (α 0, 230 ms, 280 ms)	Fit B: pre-start / hybrid only (α 0.8, 380 ms, 520 ms)
resident warm / cold	−2.8% / +20.4%	+12.8% / +33.0%	+41.3% / +56.0%
sequential + pre-start warm / cold	−30.1% / −45.4%	+8.0% / −15.4%	+8.5% / −15.3%
hybrid warm / cold	−30.3% / −40.4%	−16.9% / −28.9%	+6.8% / −8.7%
hybrid + pre-start warm / cold	−40.2% / −52.6%	−14.2% / −30.9%	−2.9% / −21.2%
sequential (unaffected)	−6.4% / −20.7%	unchanged	unchanged

Transcribed programmatically from `docs/phase-reports/PHASE_7_RESULTS_REPORT.md`, section "11.4 I-5.1".

Table 39. Deviations and unexpected findings, Part A.

#	Finding	Where
U-1	H2 fails for TTFI; in one cell pre-start is slower (+5.7%)	§5
U-2	H6 sub-claims fail: flips don't grow with batch size; sorting doesn't help; composition alone moves accuracy by up to 1.6 pt	§8
U-3	C0 fp32 lowers accuracy below all-int8 (−11% gap "recovery")	§7
U-4	No precision difference is significant at α = 0.05 (int8 vs fp32 p = 0.064)	§7
U-5	Early-exit accuracy loss is not significant (p ≥ 0.55); latency gain −19…−20% is larger than the pilot's 20-sentence look suggested (pilot: −2%)	§4
U-6	Early exit costs bytes for full-path sentences (+1.8–7%) and saves ≈ 0 over a cached session	§4.3
U-7	Engine regret 24–43% for the mid profile; time-model errors up to −52% for hybrid + pre-start	§11
U-8	The refit is a trade-off, not a fix: it would degrade resident predictions	§11.4
U-9	Cold-time predictions are pessimistic for resident plans (+24%) but optimistic for every other shape	§11.2
U-10	The visible memory peak of a pre-start plan depends on load speed (652.6 vs 813.3 MiB, same plan); the upper bound is stable	§11.1

#	Finding	Where
U-11	Native int8 differs by platform and ORT version; the aarch64 platform reproduces F-5 exactly	§9
U-12	Creation is fast only while a live WASM instance exists elsewhere	§10
U-13	Throttle bias at 50 Mbit/s (+3.2%) inflates 50 Mbit/s cold times slightly; within the gate	§1
D	Approved deviations applied: D-7.1 (E1 cache off), D-7.2 (two M4 forms), D-7.3 (accuracy per precision), D-7.4 (one calibration), D-7.5 (first-byte delay 20 ms), D-7.6 (native per platform), D-7.7 (UA spot checks). Omitted: fp32 at 5 Mbit/s (decision 4); simulated CPU throttling (F-33).	plan v1.2

Transcribed programmatically from `docs/phase-reports/PHASE_7_RESULTS_REPORT.md`, section "12. Deviations".

Appendix C Detailed Mac results (Part B)

Transcribed programmatically from `docs/phase-reports/PHASE_7_MAC_RESULTS.md` and the results report Part B. These values are never combined with Appendix B.

Table 40. Mac result files and their use.

Run	File	Used for	Status
Original, 03:36–04:39 UTC	`results/phase7/mac/f7d83e4b-phase7_mac_2026-11-01T03-36-18-093Z.json` (283 steps)	MAC-1	MAC-1 valid. Its MAC-2 timing is invalid (§5): Worker compute 6–17× slower from 03:51:37 UTC; MAC-3 / MAC-4 overlapped my aarch64 runs. Kept in full, not used for MAC-2 / 3 / 4.
MAC-5, 05:25 UTC	`results/phase7/mac/mac5_diag_fetch_transcribed.json` (your paste)	MAC-5	Valid
Re-run, 10:43–10:58 UTC	`results/phase7/mac/dcc9af05-phase7_mac_2026-11-01T10-43-02-746Z_only-MAC-2-MAC-3-MAC-4.json` (235 steps)	MAC-2, MAC-3, MAC-4	Valid

Transcribed programmatically from `docs/phase-reports/PHASE_7_MAC_RESULTS.md`, section "Phase 7: Mac Results".

Table 41. MAC-1: thread sweep (original run), baseline, w100.

Page threads	int8 M4 (ms)	fp32 M4 (ms)	int8 − fp32 [CI]	TTFI int8 / fp32 (ms)	Peak MiB int8 / fp32
1	55.72 [55.36, 56.44]	50.83 [50.59, 52.25]	+4.89 [3.22, 5.72] (+9.6%)	165 / 376	180.25 / 656.38
2	29.16 [28.94, 30.30]	26.83 [26.59, 28.26]	+2.33 [0.90, 3.47] (+8.7%)	209 / 564	180.63 / 656.75
4	16.89 [16.75, 24.06]	17.47 [17.03, 18.57]	−0.58 [−1.68, 6.58] (CI includes 0)	184 / 632	181.50 / 657.50
8	30.31 [26.82, 35.58]	25.38 [24.92, 26.99]	+4.94 [1.44, 10.20] (+19.4%)	168 / 600	183.13 / 659.06

Transcribed programmatically from `docs/phase-reports/PHASE_7_RESULTS_REPORT.md`, section "13.1 MAC-1".

Table 42. MAC-3: policies and early exit (re-run).

Plan	Peak MiB (= Part A?)	M4 off → on (ms)	Early-exit Δ [CI]	TTFI off → on (ms)
resident int8	214.06 (=)	54.99 → 43.31	−11.67 [−13.37, −8.75] (−21.2%)	502 → 430
sequential G 32 int8	80.46 (=)	90.80 → 79.70 (amort.)	−11.09 [−12.54, −10.40] (−12.2%)	891 → 676
resident mixedC2	355.06 (=)	53.88 → 42.59	−11.29 [−13.13, −10.01] (−21.0%)	536 → 466

Plan	Peak MiB (= Part A?)	M4 off → on (ms)	Early-exit Δ [CI]	TTFI off → on (ms)
sequential G 32 mixedC2	189.53 (=)	90.28 → 79.82 (amort.)	−10.46 [−12.85, −7.43] (−11.6%)	938 → 721

Transcribed programmatically from `docs/phase-reports/PHASE_7_RESULTS_REPORT.md`, section "13.2 MAC-3".

Table 43. MAC-2: decisions compared with the cloud's.

Profile	Early exit	Mac choice (warm and cold)	Cloud choice
low	off / on	int8 sequential G 32	same
mid	off	mixedC2 sequential G 32 + pre-start	same
mid	on	mixedC2 sequential G 32 + pre-start + early exit	mixedC2 resident + early exit (the near-tie of F-35)
high	off / on	fp32 resident	same

Transcribed programmatically from `docs/phase-reports/PHASE_7_MAC_RESULTS.md`, section "5.1 Decisions".

Table 44. MAC-2: regret and rank (top-4 predicted plans only).

Decision	Chosen = measured best?	Regret	Spearman ρ (pred vs measured)
all 12 (3 profiles × early exit × warm / cold)	yes, rank 1 in every case	0%	0.2–1.0

Transcribed programmatically from `docs/phase-reports/PHASE_7_MAC_RESULTS.md`, section "5.4 Regret".

Table 45. MAC-5: F-14 fetch diagnostic, six rounds.

Round	0	1	2	3	4	5
int8 ms (MB/s)	140 (480)	73 (919)	90 (749)	196 (344)	241 (279)	70 (963)
fp32 ms (MB/s)	356 (752)	257 (1,043)	280 (956)	243 (1,102)	261 (1,025)	196 (1,366)

Transcribed programmatically from `docs/phase-reports/PHASE_7_RESULTS_REPORT.md`, section "13.5 MAC-5".

Appendix D Experiment configurations

All configurations registered in `bench/configs.json` (generated from the file). Results from different configurations are never averaged together, and each result file names exactly one configuration.

Table 46. Registered configurations.

Configuration	Device; OS	Browser / runtime	Threads
CLOUD-HL-1T	x86-64, 2 vCPU, 7 GB RAM; Linux	Chromium 153 (Playwright, channel=chromium, new headless)	1
CLOUD-HL-2T	x86-64, 2 vCPU, 7 GB RAM; Linux	Chromium 153 (Playwright, channel=chromium, new headless)	2
MAC-CHROME-4T	Apple Silicon Mac (arm64), navigator.hardwareConcurrency=8, navigator.deviceMemory=16; macOS 26.7.1	Google Chrome 155 (headed, default flags)	4
MAC-CHROME-W1	Apple Silicon Mac, as MAC-CHROME-4T	Google Chrome 155 (headed, default flags)	1
CLOUD-HL-1T-20MBPS	x86-64, 2 vCPU, 7 GB RAM	Chromium 153 (Playwright, channel=chromium, persistent profile)	1
CLOUD-HL-1T-NET-50	x86-64 Intel Xeon (model not exposed), 2 vCPU, AVX2/AVX-512/AVX-512-VNNI, 7 GiB RAM; Linux 6.18 (VM)	Chromium 153.0.8010.12 (Playwright 1.63.0, channel=chromium, headless)	1
CLOUD-HL-1T-NET-20	x86-64 Intel Xeon (model not exposed), 2 vCPU, AVX2/AVX-512/AVX-512-VNNI, 7 GiB RAM; Linux 6.18 (VM)	Chromium 153.0.8010.12 (Playwright 1.63.0, channel=chromium, headless)	1
CLOUD-HL-1T-NET-10	x86-64 Intel Xeon (model not exposed), 2 vCPU, AVX2/AVX-512/AVX-512-VNNI, 7 GiB RAM; Linux 6.18 (VM)	Chromium 153.0.8010.12 (Playwright 1.63.0, channel=chromium, headless)	1
CLOUD-HL-1T-NET-5	x86-64 Intel Xeon (model not exposed), 2 vCPU, AVX2/AVX-512/AVX-512-VNNI, 7 GiB RAM; Linux 6.18 (VM)	Chromium 153.0.8010.12 (Playwright 1.63.0, channel=chromium, headless)	1
MAC-CHROME-T1	Apple Silicon Mac, as MAC-CHROME-4T	Google Chrome (headed, default flags; version recorded in the results JSON)	1
MAC-CHROME-T2	Apple Silicon Mac, as MAC-CHROME-4T	Google Chrome (headed, default flags; version recorded in the results JSON)	2
MAC-CHROME-T4	Apple Silicon Mac, as MAC-CHROME-4T	Google Chrome (headed, default flags; version recorded in the results JSON)	4
MAC-CHROME-T8	Apple Silicon Mac, as MAC-CHROME-4T	Google Chrome (headed, default flags; version recorded in the results JSON)	8
NODE-WASM-X86		onnxruntime-web 1.20.1 WASM in Node 22.23, 1 thread	—
NATIVE-X86-ORT1.20.1		Python onnxruntime 1.20.1 CPU EP, x86-64 (AVX-512-VNNI), 1 intra-op thread	—
NATIVE-X86-ORT1.29.0		Python onnxruntime 1.29.0 CPU EP, x86-64 (supplementary version row)	—

Configuration	Device; OS	Browser / runtime	Threads
NATIVE-ARM64-ORT1.20.1		Python onnxruntime 1.20.1 CPU EP (Phase 1 venv), aarch64 (asimddp), 1 intra-op thread	—

Appendix E Raw-data index

All raw data are preserved in the repository. Nothing was deleted, including failed attempts (kept as rows with `retryOf`), pilot and smoke-test data, and the invalid part of the first Mac run.

Table 47. Phase 7 raw-data index (results report §17).

Data	Location
Part A browser runs (1,310)	`results/phase7/{acc,ebrowser,e1,f28,eng,e3bytes,netcheck,net,e5x,g1}/runs.jsonl`; `eng/{calibration_cloud,decisions,decision_traces}.json`; `e3bytes/selection.json`; per-invocation `env_*.json`
Part A offline passes (249)	`results/phase7/wasm/` (90), `native-x86-ort1.20.1/` (77), `native-arm64-ort1.20.1/` (77), `native-x86-ort1.29.0/` (5)
Part B Mac	`results/phase7/mac/` (original run, re-run, MAC-5 transcription)
Analyses	`results/phase7/summary/analysis.json` (Part A), `analysis_mac.json` (original Mac run alone), `analysis_mac_final.json` (Part B final)
Pilot (not results)	`results/phase7/pilot/`
Smoke / self-tests (not results)	`results/phase7/_smoke/`
Workload	`results/phase7/workload_100.json`

Transcribed programmatically from `docs/phase-reports/PHASE_7_RESULTS_REPORT.md`, section "17. Raw-data index".

Table 48. File counts in `results/phase7/` at the time of writing (proposed Phase 7 commit).

Folder	Files	Folder	Files
`net` (incl. 80 `profile_*.txt` throttling logs)	97	`eng`	8
`wasm`	90	`e1`, `e3bytes`, `native-x86-ort1.29.0`	5 each
`native-x86-ort1.20.1`	77	`acc`, `f28`	4 each
`native-arm64-ort1.20.1`	77	`ebrowser`, `g1`, `mac`, `netcheck`, `summary`	3 each
`_smoke` (not results)	43	`workload_100.json`	1
`e5x`	39	Total	494
`pilot` (not results)	24		

Earlier phases' raw results are in `results/phase1` to `results/phase6` and are referenced from their phase reports.

Appendix F Repository structure and reproduction

Table 49. Repository structure (top level).

Path	Contents
`python/`	Model pipeline (`run_phase1.sh`, split, export, quantize, exit heads, validate, manifest) and diagnostics
`models/onnx/`	`manifest.json`, `early_exit.json`, `vocab.txt` (tracked); ONNX files (generated, not tracked)
`packages/adaptive-onnx-loader/`	The package: `src/`, `test/`, `index.d.ts`, `README.md`, `LICENSE` (MIT)
`demo/`	`server.mjs` and the demonstration pages for Phases 2–6
`bench/`	Browser tests per phase, Phase 7 harness and analysis (`bench/phase7/`), `configs.json`
`results/`	Raw results and analyses per phase
`docs/`	Master plan, findings register, experiment plan, phase reports, this report (`docs/final-report/`)
`scripts/fetch_resources.sh`	Downloads the checkpoint and SST-2 files

Reproduction outline (commands as documented in the phase reports):

1. `scripts/fetch_resources.sh`: download the checkpoint and data (needs access to huggingface.co).
2. `python/run_phase1.sh`: rebuild the ONNX components, exit heads and manifest.
3. `npm test` in `packages/adaptive-onnx-loader`: unit tests (40).
4. `node demo/server.mjs`, then open `http://localhost:8080/` for the demonstration page.
5. `node bench/phase7/suite.mjs` and `node bench/phase7/validate.mjs <block>` for the cloud experiments; `node bench/phase7/analyze.mjs` regenerates `results/phase7/summary/analysis.json` from the raw files.
6. For the report itself: `docs/final-report/tools/make_charts.py` regenerates the charts from the summary files; `capture_screenshots.mjs` regenerates the documentation screenshots; `src/build.js` builds this document.